CHAKOTEAM! BOOKHAND

General
Template 2

Grafos
Bellman Ford 2
Binary Lifting 2
Djikstra 2
Dominator Tree 2
Floyd Warshal 3
Kruskal 3
Lca 3

Flujo Y Conectividad
2-sat 3
Camino Dirigido Euler 4
Camino No Dirigido Euler 4
Dinic 4
Tarjan 5
TarjanSCC 5

Estructuras De Datos
Fenwick 5
Fenwick 2D 5
Fenwick Walk 5
Line Container 6
Segment Tree 6
Segment Tree Lazy 6
Segment Tree Misra Gries 6
Segment Tree Walk 7
Sparse Table 7
Sqrt Decomposition 7
Union Find 7

Matematicas
Bin Pow 7
Catalan Numbers 7
Convex Hull 7
Crt 8
Discrete Log 8
Discrete Root 8
Erathostenes Sieve 8
Factorization Pollard Rho 8
Fft 9
Gaussian Elimination 9
Gcd 9
Gcd Extended 9
Inclusion Exclusion 9
Miller Rabin 9
Mobius 10
Ntt 10
Primitive Root 10
Simpson Rule 10
Template 10
Tonelli Shanks 11
Tot Euler 11
Z Transform 11

Strings
Aho Corasick 11
Booth 12
Kmp 12
Manacher 12
Rolling Hashing 12
Suffix Array 12
Trie 12
Zfunction 13

Programacion Dinamica
Knappsack 13
Lis 13
Lis Length K 13

Offline Algorithms
Cdq 13
Mo 13
Pbs 14
Value Driven 14

Miscelaneo
Busqueda Binaria 14
Golden Search 14
Random 14
Time 14
Xorshift 14

Geometria
Circle-intersection 15
Circle-polygon-intersection 15
Circle-tangents 15
Circle 15
Convex-hull-3d 15
Convex-hull 16
Delaunay-graph 16
Delaunay 16
Halfplane-intersection 17
Halfplane 17
Hull-diameter 17
In-convex 18
Lattice-point 18
Line-distance 18
Minimum-enclosing-circle 18
Minimum-enclosing-sphere 18
Minkowski 18
Nearest-2-points 18
Order-by-angle 19
Order-by-slope 19
Point-2d 19
Point-3d 19
Point-inside-polygon 20
Polygon-area 20
Polygon-center 20
Polygon-cut 20
Segment-distance 20
Segment 20
Sphere 20

Formulario
01 Notacion 21
02 Teoria De Numeros 21
03 Combinatoria 21
04 Recurrencias Y Matrices 22
05 Fracciones Continuas 22
06 Polinomios Bezout Sylvester 22
07 Series Formales E Hipergeometricas 22
08 Funciones Generatrices 22
09 Grupos Burnside Polya 23
10 Geometria Computacional 23
11 Probabilidad 24
12 Algebra Lineal Y Otros 24

General

Template

```cpp
#pragma GCC optimize("O3", "unroll-loops")
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define endl '\n'
#define all(v) (v).begin(), (v).end()

void solve(){

}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll t = 1; cin >> t;
    while(t--) solve();
}
```

Grafos

Bellman Ford

```cpp
struct BellmanFord {
  struct Edge {
    int from, to, weight;
  };
  int n, last_updated = -1;
  const ll INF = 1e18;
  vector<int> p, dist;
  BellmanFord(vector<Edge>& G, int s, int nn) {
    n = nn;
    dist.assign(n + 2, INF);
    p.assign(n + 2, -1);
    dist[s] = 0;
    for (int i = 1; i <= n; i++) {
      last_updated = -1;
      for (Edge& e : G)
        if (dist[e.from] + e.weight < dist[e.to]) {
          dist[e.to] = dist[e.from] + e.weight;
          p[e.to] = e.from;
          last_updated = e.to;
        }
    }
  }
  bool getCycle(vector<int>& cycle) {
    if (last_updated == -1) return false;
    for (int i = 0; i < n - 1; i++) last_updated = p[last_u
pdated];
    for (int x = last_updated;; x = p[x]) {
      cycle.push_back(x);
      if (x == last_updated and cycle.size() > 1) break;
    }
    reverse(cycle.begin(), cycle.end());
    return true;
  }
};
```

Binary Lifting

```cpp
void BinaryLifting(ll node) {
  stack<pair<ll, ll>> st;
  st.push({node, -1});
  while (st.size()) {
    auto [n, p] = st.top();
    st.pop();
    for (auto v : tree[n]) {
      if (p == v) continue;
      st.push({v, n});
      bl[v][0] = n;
      for (ll j = 1; j < 33; j++) {
        if (bl[v][j - 1] != -1) {
          bl[v][j] = bl[bl[v][j - 1]][j - 1];
        } else {
          bl[v][j] = -1;
        }
      }
    }
  }
}
```

Djikstra

```cpp
vector<ll> dist;
vector < vector<pair<ll, ll>> g;

void djikstra() {
  priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greate
r<pair<ll, ll>>> pq;
  pq.push({0, 0});  // nodo_inicio, distancia
  while (pq.size()) {
    auto [c, node] = pq.top();
```

Djikstra (cont.)

```cpp
    pq.pop();
    if (dist[node] <= c) continue;
    dist[node] = c;
    for (auto [c_, v_] : g[node]) {
      if (dist[v_] > c + c_) pq.push({c + c_, v_});
    }
  }
}
```

Dominator Tree

```cpp
/*
  *Description:* Computes the inmediate dominator of every
node in a directed graph in O((|E|+|V|)log(|V|)).
  *Status:* Tested on CSES
*/
struct dominator_tree {
    ll n,l,s;
    vector<ll> tin,par,vis_ord, dsu_mn, dsu_par, sdom, dep,
rdom,idom;
    vector<bool> vis;
    vector<vector<ll>> up;
    vector<vector<pair<ll,ll>>> up_mn;
    void dfs(ll u, ll p, vector<vector<ll>> & adj){
        if (vis[u]) return;
        vis[u] = 1;
        dep[u] = dep[p]+1;
        tin[u] = vis_ord.size();
        vis_ord.push_back(u);
        par[u] = p;
        for (int v : adj[u]) dfs(v,u,adj);
    }
    ll find(ll x){
        if (dsu_par[x] == -1) return x;
        if (dsu_par[dsu_par[x]] != -1){
            ll res = find(dsu_par[x]);
            if (tin[res] < tin[dsu_mn[x]])
                dsu_mn[x] = res;
            dsu_par[x] = dsu_par[dsu_par[x]];
        }
        assert(dsu_mn[x] != INT32_MAX);
        return dsu_mn[x];
    }
    void get_sdom(vector<vector<ll>> & rev){
        for (ll i = vis_ord.size()-1; i >= 0; i--){
            ll c = vis_ord[i];
            if (c == s) continue;
            for (ll to : rev[c]){
                ll res = find(to);
                if (sdom[c] == -1 || tin[res] < tin[sdom[c]
])
                    sdom[c] = res;
            }
            dsu_par[c] = par[c];
            dsu_mn[c] = sdom[c];
            if (sdom[c] == -1) cerr << c << endl;
        }
    }
    void init_up(){
        for_(i, n){
            if (!vis[i]) continue;
            up[i][0] = par[i];
            up_mn[i][0] = {tin[sdom[i]],i};
        }
        for (ll i = 1; i < l; i++) {
            for_(j, n) {
                if (!vis[j]) continue;
                ll h = up[j][i-1];
                if (h == -1) {
                    up[j][i] = -1;
                    up_mn[j][i] = up_mn[j][i-1];
                } else {
                    up[j][i] = up[h][i-1];
                    up_mn[j][i] = min(up_mn[j][i-1], up_mn[
h][i-1]);
                }
            }
        }
    }
    ll get_min(ll v, ll u){
        ll dist = dep[u]-dep[v];
        pair<ll,ll> ret = up_mn[u][0];
        ll w = u;
        for_(i, l)
            if (dist&(1<<i)){
                ret = min(ret,up_mn[w][i]);
                w = up[w][i];
            }
        assert(w == v);
        return ret.second;
    }
    void get_idom(){
        for (ll c : vis_ord){
            if (c == s) continue;
            rdom[c] = get_min(sdom[c],c);
```

Dominator Tree (cont.)

```
        }
        for (ll c : vis_ord)
            if (c != s)
                idom[c] = ((rdom[c] == c)?sdom[c]:idom[rdom[c]]);
    }
    dominator_tree(vector<vector<ll>> & adj, vector<vector<ll>> & rev, ll _s = 0, ll logn = 20){
        n = adj.size(); l = logn; s = _s;
        tin.resize(n,INT32_MAX);
        par.resize(n,-1);
        vis.resize(n,0);
        dep.resize(n,0);
        dfs(s,-1,adj);
        dsu_par.resize(n,-1);
        dsu_mn.resize(n,INT32_MAX);
        sdom.resize(n,-1);
        rdom.resize(n,-1);
        idom.resize(n,-1);
        get_sdom(rev);
        up.assign(n,vector<ll>(logn));
        up_mn.assign(n,vector<pair<ll,ll>>(logn));
        init_up();
        get_idom();
    }
};

//aux function, cover dom tree
void dfs(ll &cnt, ll node, vector<vector<ll>> &adj, vector<ll> &tin, vector<ll> &tout){
    tin[node] = cnt++;
    for(ll child : adj[node]) dfs(cnt, child, adj, tin, tout);
    tout[node] = cnt++;
}

//verifiy if u dominates v, for each edge, obviusly tle, is to catch de idea, and implement a part of the function
vector<vector<bool>> Verifiy_u_dom_v(ll n, vector<vector<ll>> &adj, vector<vector<ll>> &rev_adj, vector<pair<ll, ll>> &edges){
    dominator_tree dt(adj, rev_adj);
    vector<vector<ll>> dom_tree(n);
    for_(i, n){
        if(i==0) continue;
        if(dt.vis[i]) dom_tree[dt.idom[i]].push_back(i);
    }
    vector<ll> tin(n, 0), tout(n, 0); ll cnt = 0;
    dfs(cnt, 0, dom_tree, tin, tout);
    vector<vector<bool>> dom_matrix(n, vector<bool>(n, false));
    for_(i, edges.size()){
        auto &[u,v] = edges[i];
        if((tin[u]<=tin[v] && tout[v]<=tout[u])) dom_matrix[u][v] = true;
    }
    return dom_matrix;
}
```

Floyd Warshal

```
for (int i = 1; i <= n; i++) {
  for (int j = 1; j <= n; j++) {
    if (i == j)
      distance[i][j] = 0;
    else if (adj[i][j])
      distance[i][j] = adj[i][j];
    else
      distance[i][j] = INF;
  }
}
for (int k = 1; k <= n; k++) {
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      distance[i][j] = min(distance[i][j], distance[i][k] + distance[k][i]);
    }
  }
}
```

Kruskal

```
struct Edge {
  int a;
  int b;
  ll w;
  Edge(int a_, int b_, ll w_) : a(a_), b(b_), w(w_) {}
};
bool c_edge(Edge& a, Edge& b) { return a.w < b.w; }
ll Kruskal() {
  int n = G.size();
  union_find sets(n);
  vector<Edge> edges;
  for (int i = 0; i < n; i++) {
    for (auto eg : G[i]) {
      // node i to node eg.first with cost eg.second
      edges.emplace_back(i, eg.first, eg.second);
    }
  }
  sort(edges.begin(), edges.end(), c_edge);
  ll min_cost = 0;
  for (Edge e : edges) {
    if (sets.sameSet(e.a, e.b) != true) {
      tree.emplace_back(e.a, e.b, e.w);
      min_cost += e.w;
      sets.unionSet(e.a, e.b);
    }
  }
  return min_cost;
}
```

Kruskal (cont.)

Lca

```
struct LCA {
  vector<vector<ll>> T, parent;
  vector<ll> depth, vis, comp;
  ll LOGN, V;
  LCA(vector<vector<ll>>& T) : T(T) {
    V = (ll)T.size();
    LOGN = 1;
    while ((1LL << LOGN) <= max(1LL, V)) LOGN++;
    parent.assign(V, vector<ll>(LOGN, 0));
    depth.assign(V, 0);
    vis.assign(V, 0);
    comp.assign(V, -1);
    ll id = 0;
    for (ll u = 0; u < V; u++) {
      if (!vis[u]) {
        dfs(u, u, id++);
      }
    }
  }

  void dfs(ll u, ll p, ll id) {
    vis[u] = 1;
    comp[u] = id;
    parent[u][0] = p;
    for (ll j = 1; j < LOGN; j++)
      parent[u][j] = parent[parent[u][j - 1]][j - 1];

    for (ll v : T[u]) {
      if (!vis[v]) {
        depth[v] = depth[u] + 1;
        dfs(v, u, id);
      }
    }
  }
  ll query(ll u, ll v) {
    if (comp[u] != comp[v]) return -1;
    if (depth[u] < depth[v]) swap(u, v);
    ll k = depth[u] - depth[v];
    for (ll j = LOGN - 1; j >= 0; j--) {
      if ((k >> j) & 1) u = parent[u][j];
    }
    if (u == v) return u;
    for (ll j = LOGN - 1; j >= 0; j--) {
      if (parent[u][j] != parent[v][j]) {
        u = parent[u][j];
        v = parent[v][j];
      }
    }
    return parent[u][0];
  }
};
```

Flujo Y Conectividad

2-sat

```
struct TwoSat {
  int N;
  vector<vector<int>> gr;
  vector<int> values;
  TwoSat(int n = 0) : N(n), gr(2 * n) {}
  int addVar() {
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }
  void either(int f, int j) {
    f = max(2 * f, -1 - 2 * f);
    j = max(2 * j, -1 - 2 * j);
    gr[f].push_back(j ^ 1);
    gr[j].push_back(f ^ 1);
  }
  void setValue(int x) { either(x, x); }
  void atMostOne(const vector<int>& li) {
    if ((int)li.size() <= 1) return;
```

2-sat (cont.)

```
    int cur = ~li[0];
    for (int i = 2; i < (int)li.size(); i++) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }
  vector<int> val, comp, z;
  int Time = 0;
  int dfs(int i) {
    int low = val[i] = ++Time, x;
    z.push_back(i);
    for (int e : gr[i])
      if (!comp[e]) low = min(low, val[e] ? val[e] : dfs(e)
);
    if (low == val[i]) do {
        x = z.back();
        z.pop_back();
        comp[x] = low;
        if (values[x >> 1] == -1) values[x >> 1] = x & 1;
      } while (x != i);
    return val[i] = low;
  }
  bool solve() {
    values.assign(N, -1);
    val.assign(2 * N, 0);
    comp = val;
    for (int i = 0; i < 2 * N; i++)
      if (!comp[i]) dfs(i);
    for (int i = 0; i < N; i++)
      if (comp[2 * i] == comp[2 * i + 1]) return false;
    return true;
  }
};
```

Camino Dirigido Euler

```
struct directed_euler_trail {
  ll n, m = 0;
  vector<vector<ll>> adj;
  vector<pair<ll, ll>> E;
  vector<ll> indeg, outdeg, vertices, edges;
  directed_euler_trail(ll n) : n(n), adj(n), indeg(n), outd
eg(n) {}
  void add_edge(ll u, ll v) {
    adj[u].push_back(m);
    E.push_back({u, v});
    outdeg[u]++, indeg[v]++, m++;
  }
  ll find_start() {
    ll s = -1, cp = 0, cm = 0;
    for (ll v = 0; v < n; v++) {
      ll d = outdeg[v] - indeg[v];
      if (d == 1)
        cp++, s = v;
      else if (d == -1)
        cm++;
      else if (d != 0)
        return -2;
      if (outdeg[v] > 0 && s < 0) s = v;
    }
    if (cp == 0 && cm == 0) return s;
    return (cp == 1 && cm == 1) ? s : -2;
  }
  bool solve() {
    if (m == 0) {
      vertices = {0};
      return true;
    }
    ll s = find_start();
    if (s == -2) return false;
    vector<ll> ep(n);
    edges.resize(m);
    ll pp = 0, pq = m, v = s;
    while (pp < pq) {
      if (ep[v] >= (ll)adj[v].size()) {
        if (pp == 0) return false;
        edges[--pq] = edges[--pp];
        v = E[edges[pq]].first;
      } else {
        ll e = adj[v][ep[v]++];
        edges[pp++] = e;
        v = E[e].second;
      }
    }
    vertices = {s};
    for (ll e : edges) vertices.push_back(E[e].second);
    return true;
  }
};
```

Camino No Dirigido Euler

```
struct undirected_euler_trail {
  ll n, m = 0;
  vector<vector<ll>> adj;
  vector<pair<ll, ll>> E;
  vector<ll> deg, vertices, edges;

  undirected_euler_trail(ll n) : n(n), adj(n), deg(n) {}

  void add_edge(ll u, ll v) {
    adj[u].push_back(m);
    if (u != v) adj[v].push_back(m);
    E.push_back({u, v});
    deg[u]++, deg[v]++, m++;
  }

  ll find_start() {
    ll s = -1, codd = 0;
    for (ll v = 0; v < n; v++) {
      if (deg[v] & 1)
        codd++, s = v;
      else if (deg[v] > 0 && s < 0)
        s = v;
    }
    return (codd == 0 || codd == 2) ? s : -2;
  }

  bool solve() {
    if (m == 0) {
      vertices = {0};
      return true;
    }

    ll s = find_start();
    if (s == -2) return false;

    vector<ll> ep(n);
    edges.resize(m);
    vector<bool> alive(m, true);
    ll pp = 0, pq = m, v = s;

    while (pp < pq) {
      if (ep[v] >= adj[v].size()) {
        if (pp == 0) return false;
        edges[--pq] = edges[--pp];
        v ^= E[edges[pq]].first ^ E[edges[pq]].second;
      } else {
        ll e = adj[v][ep[v]++];
        if (!alive[e]) continue;
        alive[e] = false;
        edges[pp++] = e;
        v ^= E[e].first ^ E[e].second;
      }
    }
    vertices = {s};
    for (ll e : edges) {
      s ^= E[e].first ^ E[e].second;
      vertices.push_back(s);
    }
    return true;
  }
};
```

Dinic

```
template <class T>
struct Dinic {
  struct Edge {
    int to, rev;
    T f, c;
    bool r;
  };
  int n, t_;
  vector<vector<Edge>> G;
  vector<pair<ll, ll>> min_cut;
  vector<int> D, q, W;
  bool bfs(int s, int t) {
    W.assign(n, 0);
    D.assign(n, -1);
    D[s] = 0;
    int f = 0, l = 0;
    q[l++] = s;
    while (f < l) {
      int u = q[f++];
      for (const Edge& e : G[u])
        if (D[e.to] == -1 && e.f < e.c) D[e.to] = D[u] + 1,
 q[l++] = e.to;
    }
    return D[t] != -1;
  }
  int dfs(int u, T f) {
    if (u == t_) return f;
    for (int& i = W[u]; i < (int)G[u].size(); ++i) {
      Edge& e = G[u][i];
      int v = e.to;
      if (e.c <= e.f || D[v] != D[u] + 1) continue;
```

Dinic (cont.)

```
      T df = dfs(v, min(f, e.c - e.f));
      if (df > 0) {
        e.f += df, G[v][e.rev].f -= df;
        return df;
      }
    }
    return 0;
  }
  Dinic(int N) : n(N), G(N), D(N), q(N) {}
  void add_edge(int u, int v, T cap) {
    G[u].push_back({v, (int)G[v].size(), 0, cap, 0});
    G[v].push_back({u, (int)G[u].size() - 1, 0, 0, 1});  // Use cap instead of 0 if bidirectional
  }
  T max_flow(int s, int t) {
    t_ = t;
    T ans = 0;
    while (bfs(s, t))
      while (T dl = dfs(s, numeric_limits<T>::max())) ans += dl;
    return ans;
  }
  void solve_min_cut(vector<pair<ll, ll>>& edges) {
    for (auto [u, v] : edges) {
      if ((D[u] != -1) != (D[v] != -1)) {
        min_cut.emplace_back(u, v);
      }
    }
  }
};
```

Tarjan

```
struct tarjan {
  struct edge {
    int u, v, comp, bridge;
  };
  int n, nbc, T;
  vector<vector<int>> adj;
  vector<edge> e;
  vector<int> disc, low, art, st;
  tarjan(int n) : n(n), nbc(0), T(0), adj(n), disc(n, -1), low(n), art(n, 0) {}
  int add_edge(int u, int v) {
    int i = e.size();
    adj[u].push_back(i);
    adj[v].push_back(i);
    e.push_back({u, v, -1, false});
    return i;
  }
  void dfs(int u, int pe) {
    low[u] = disc[u] = T++;
    for (int i : adj[u]) {
      if (i == pe) continue;
      int v = e[i].u ^ e[i].v ^ u;
      if (disc[v] < 0) {
        st.push_back(i);
        dfs(v, i);
        if (low[v] > disc[u]) e[i].bridge = true;
        if (low[v] >= disc[u]) {
          art[u]++;
          while (st.back() != i) {
            e[st.back()].comp = nbc;
            st.pop_back();
          }
          e[st.back()].comp = nbc;
          st.pop_back();
          nbc++;
        }
        low[u] = min(low[u], low[v]);
      } else if (disc[v] < disc[u]) {
        st.push_back(i);
        low[u] = min(low[u], disc[v]);
      }
    }
  }
  void solve() {
    for (int i = 0; i < n; i++)
      if (disc[i] < 0) dfs(i, -1), art[i]--;
  }
  bool is_bridge(int i) { return e[i].bridge; }
  bool is_art(int v) { return art[v] > 0; }
  int component(int i) { return e[i].comp; }
};
```

TarjanSCC

```
struct tarjan {
  int n, timer;
  vector<bool> vis, ist;
  vector<int> tin, low, st, cmp;
  vector<vector<int>> scc;
  void dfs(int u, vector<vector<int>>& adj) {
    vis[u] = ist[u] = 1;
    st.push_back(u);
    tin[u] = low[u] = timer++;
    for (auto v : adj[u]) {
      if (!vis[v]) {
        dfs(v, adj);
        low[u] = min(low[u], low[v]);
      } else if (ist[v]) {
        low[u] = min(low[u], tin[v]);
      }
    }
    if (low[u] == tin[u]) {
      scc.emplace_back();
      int cur;
      do {
        cur = st.back();
        st.pop_back();
        ist[cur] = 0;
        scc.back().push_back(cur);
        cmp[cur] = (int)scc.size() - 1;
      } while (cur != u);
    }
  }
  tarjan(vector<vector<int>>& adj) {
    n = adj.size();
    tin.resize(n);
    timer = 0;
    cmp.resize(n);
    low.resize(n);
    vis.resize(n, 0);
    ist.resize(n, 0);
    for (int i = 0; i < n; i++)
      if (!vis[i]) dfs(i, adj);
  }
};
```

Estructuras De Datos

Fenwick

```
struct fenwick_tree {
  vector<ll> bit;
  int n;
  fenwick_tree(int n) : n(n) { bit.assign(n, 0); }
  fenwick_tree(vector<ll>& a) : fenwick_tree(a.size()) {
    for (size_t i = 0; i < a.size(); i++) {
      bit[i] += a[i];
      if ((i | (i + 1)) < a.size()) bit[i | (i + 1)] += bit[i];
    }
  }
  ll sum(int r) {
    ll ret = 0;
    for (; r >= 0; r = (r & (r + 1)) - 1) ret += bit[r];
    return ret;
  }
  ll sum(int l, int r) { return sum(r) - sum(l - 1); }
  void add(int idx, ll delta) {
    for (; idx < n; idx = idx | (idx + 1)) bit[idx] += delta;
  }
};
```

Fenwick 2D

```
struct fenwick_tree_2d {
  int N, M;
  vector<vector<ll>> BIT;
  fenwick_tree_2d(int N, int M) : N(N), M(M) {
    BIT.assign(N + 1, vector<ll>(M + 1, 0));
  }
  void update(int x, int y, ll v) {
    for (int i = x; i <= N; i += (i & -i))
      for (int j = y; j <= M; j += (j & -j)) BIT[i][j] += v;
  }
  ll sum(int x, int y) {
    ll s = 0;
    for (int i = x; i > 0; i -= (i & -i))
      for (int j = y; j > 0; j -= (j & -j)) s += BIT[i][j];
    return s;
  }
  ll query(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) +
           sum(x1 - 1, y1 - 1);
  }
};
```

Fenwick Walk

```
struct fenwick_tree_with_walk : fenwick_tree {
  ll walk(ll k) {
    if (k <= 0 || k > sum(n - 1)) return n;
    ll i = -1, s = 1;
```

Fenwick Walk (cont.)

```
    while ((s << 1) < n) s <<= 1;
    while (s > 0) {
      ll m = i + s;
      if (m < n && bit[m] < k) {
        i = m;
        k -= bit[m];
      }
      s >>= 1;
    }
    return i + 1;
  }
};
```

Line Container

```
const ll INF = 1e15;
struct Line {
  mutable ll a, b, c;
  bool operator<(Line r) const {
    return a < r.a;
  }
  bool operator<(ll x) const {
    return c < x;
  }
};
// dynamically insert `a*x + b` lines and query for maximum
// at any x all operations have complexity O(log N)
struct LineContainer: multiset < Line, less < >> {
  ll div(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x -> c = INF, 0;
    if (x -> a == y -> a) x -> c = x -> b > y -> b ? INF : -INF;
    else x -> c = div(y -> b - x -> b, x -> a - y -> a);
    return x -> c >= y -> c;
  }

  void add(ll a, ll b) {
    // a *= -1, b *= -1 // for min
    auto z = insert({
      a,
      b,
      0
    }), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x) -> c >= y -> c) isect(x, erase(y));
  }

  ll query(ll x) {
    if (empty()) return -INF; // INF for min
    auto l = * lower_bound(x);
    return l.a * x + l.b;
    // return -l.a * x - l.b; // for min
  }
};
```

Segment Tree

```
template<class T, T m_(T, T)> struct segment_tree{
  int n; vector<T> ST;
  segment_tree(){}
  segment_tree(vector<T> &a){
    n = a.size(); ST.resize(n << 1);
    for (int i=n;i<(n<<1);i++)ST[i]=a[i-n];
    for (int i=n-1;i>0;i--)ST[i]=m_(ST[i<<1],ST[i<<1|1]);
  }
  void update(int pos, T val){ // replace with val
    ST[pos += n] = val;
    for (pos >>= 1; pos > 0; pos >>= 1)
      ST[pos] = m_(ST[pos<<1], ST[pos<<1|1]);
  }
  T query(int l, int r){ // [l, r]
    T ansL, ansR; bool hasL = 0, hasR = 0;
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1)
        ansL=(hasL?m_(ansL,ST[l++]):ST[l++]),hasL=1;
      if (r & 1)
        ansR=(hasR?m_(ST[--r],ansR):ST[--r]),hasR=1;
    }
    if (!hasL) return ansR; if (!hasR) return ansL;
    return m_(ansL, ansR);
  }
};
```

Segment Tree Lazy

```
template<
  class T1, // answer value stored on nodes
  class T2, // lazy update value stored on nodes
```

Segment Tree Lazy (cont.)

```
  T1 merge(T1, T1),
  void pushUpd(T2& /*parent*/, T2& /*child*/, int, int, int, int), // push update value from a node to another. parent -> child
  void applyUpd(T2&, T1&, int, int)           // apply the update value of a node to its answer value. upd -> ans
>
struct segment_tree_lazy{
  vector<T1> ST; vector<T2> lazy; vector<bool> upd;
  int n;
  void build(int i, int l, int r, vector<T1>&values){
    if (l == r){
      ST[i] = values[l];
      return;
    }
    build(i << 1, l, (l + r) >> 1, values);
    build(i << 1 | 1, (l + r) / 2 + 1, r, values);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  segment_tree_lazy() {}
  segment_tree_lazy(vector<T1>&values){
    n = values.size(); ST.resize(n << 2 | 3);
    lazy.resize(n << 2 | 3); upd.resize(n << 2 | 3, false);
    build(1, 0, n - 1, values);
  }
  void push(int i, int l, int r){
    if (upd[i]){
      applyUpd(lazy[i], ST[i], l, r);
      if (l != r){
        pushUpd(lazy[i], lazy[i << 1], l, r, l, (l + r) / 2);
        pushUpd(lazy[i], lazy[i << 1 | 1], l, r, (l + r) / 2 + 1, r);
        upd[i << 1] = 1;
        upd[i << 1 | 1] = 1;
      }
      upd[i] = false;
      lazy[i] = T2();
    }
  }
  void update(int i, int l, int r, int a, int b, T2 &u){
    if (l >= a and r <= b){
      pushUpd(u, lazy[i], a, b, l, r);
      upd[i] = true;
    }
    push(i, l, r);
    if (l > b or r < a) return;
    if (l >= a and r <= b) return;
    update(i << 1, l, (l + r) >> 1, a, b, u);
    update(i << 1 | 1, (l + r) / 2 + 1, r, a, b, u);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  void update(int a, int b, T2 u){
    if (a > b){
      update(0, b, u);
      update(a, n - 1, u);
      return ;
    }
    update(1, 0, n - 1, a, b, u);
  }
  T1 query(int i, int l, int r, int a, int b){
    push(i, l, r);
    if (a <= l and r <= b)
      return ST[i];
    int mid = (l + r) >> 1;
    if (mid < a)
      return query(i << 1 | 1, mid + 1, r, a, b);
    if (mid >= b)
      return query(i << 1, l, mid, a, b);
    return merge(query(i << 1, l, mid, a, b), query(i << 1 | 1, mid + 1, r, a, b));
  }
  T1 query(int a, int b){
    if (a > b) return merge(query(a, n - 1), query(0, b));
    return query(1, 0, n - 1, a, b);
  }
};
```

Segment Tree Misra Gries

```
template<ll K>
vector<pair<ll, ll>> misra_merge(vector<pair<ll, ll>> a, vector<pair<ll, ll>> b) {
  for (auto [x, c]: b) {
    auto it = find_if(a.begin(), a.end(), [&](auto i){return i.first == x;});
    if (it != a.end()) it->second += c;
    else a.emplace_back(x, c);
  }
  while (a.size() >= K) {
    ll mini = LLONG_MAX;
    for (auto [x, c]: a) mini = min(mini, c);
    vector<pair<ll, ll>> tmp;
    for (auto [x, c]: a) {
      c -= mini;
```

Segment Tree Misra Gries (cont.)

```
      if (c > 0) tmp.emplace_back(x, c);
    }
    a = tmp;
  }
  return a;
}
template<ll K>
struct segment_tree_misra_gries : segment_tree<vector<pair<ll, ll>>, misra_merge<K>> {
  using base = segment_tree<vector<pair<ll, ll>>, misra_merge<K>>;
  using base::base;
};
```

Segment Tree Walk

```
template<class T, T m_(T, T)> struct segment_tree_with_walk : segment_tree<T,m_> {
  int walk(int l, T k) {
    vector<int> L, R;
    int r = this->n - 1;
    for (l += this->n, r += this->n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1) L.push_back(l++);
      if (r & 1) R.push_back(--r);
    }
    reverse(R.begin(), R.end());
    L.insert(L.end(), R.begin(), R.end());
    for (int v : L) {
      if (this->ST[v] >= k) {
        while (v < this->n) v = (this->ST[v << 1] >= k) ? (v << 1) : (v << 1 | 1);
        return v - this->n;
      }
    }
    return -1;
  }
};
```

Sparse Table

```
template <typename T, T m_(T, T)>
struct sparse_table {
  int n;
  vector<vector<T>> table;
  sparse_table() {}
  sparse_table(vector<T> &arr) {
    n = arr.size();
    int k = log2_floor(n) + 1;
    table.assign(n, vector<T>(k));
    for (int i = 0; i < n; i++)
      table[i][0] = arr[i];
    for (int j = 1; j < k; j++)
      for (int i = 0; i + (1 << j) <= n; i++)
        table[i][j] = m_(table[i][j-1], table[i+(1<<(j-1))][j-1]);
  }
  T query(int l, int r) {
    int k = log2_floor(r - l + 1);
    return m_(table[l][k], table[r-(1 << k)+1][k]);
  }
  int log2_floor(int n) { return n ? __builtin_clzll(1) - __builtin_clzll(n) : -1; }
};
```

Sqrt Decomposition

```
template <class T, T merge(T,T)>
struct sqrt_decomp {
  int n,sq;
  vector<T> st,bl;
  sqrt_decomp(vector<T> & a){
    n = a.size(); st.resize(n);
    sq = sqrtl(n); bl.resize((n+sq-1)/sq);
    for (int i = 0; i < n; i++){
      st[i] = a[i];
      bl[i/sq] = ((i%sq == 0)?st[i]:merge(bl[i/sq],st[i]));
    }
  }
  void update(int pos, T val) {
    st[pos] = val;
    int block = pos/sq, lb = block*sq, rb = min(block*sq+sq-1,n-1);
    T vl;
    for (int i = lb; i <= rb; i++)
      vl = ((i == lb)?st[i]:merge(vl,st[i]));
    bl[block] = vl;
  }
  T query(int l, int r){
    T res; bool hasR = 0;
    for (int i = l/sq; i <= r/sq; i++){
      int lb = i*sq, rb = i*sq+sq-1;
      if (l <= lb && rb <= r){
        res = (hasR?merge(res,bl[i]):bl[i]), hasR = 1;
        continue;
      }
      for (int j = max(l,lb); j <= min(rb,r); j++)
        res = (hasR?merge(res,st[j]):st[j]), hasR = 1;
    }
    return res;
  }
};
```

Sqrt Decomposition (cont.)

Union Find

```
struct union_find {
  vector<int> e;
  union_find(int n) { e.assign(n, -1); }
  int findSet(int x) { return (e[x] < 0 ? x : e[x] = findSet(e[x])); }
  bool sameSet(int x, int y) { return findSet(x) == findSet(y); }
  int size(int x) { return -e[findSet(x)]; }
  bool unionSet(int x, int y) {
    x = findSet(x), y = findSet(y);
    if (x == y) return 0;
    if (e[x] > e[y]) swap(x, y);
    e[x] += e[y], e[y] = x;
    return 1;
  }
};
```

Matematicas

Bin Pow

```
ll binpow(ll a, ll b, ll m) {
  ll ans = 1;
  while (b > 0) {
    if (b & 1) ans = (ans * a) % m;
    a = (a * a) % m;
    b >>= 1;
  }
  return ans;
}
```

Catalan Numbers

```
// n-catalan number = (2n)!/((n+1)!n!)
ll getNCatalanNumber(ll n) {
  double ans = 1;
  for (ll k = 2; k <= n; k++) ans *= (n + k) / k;
  return (ll)(ans + 0.001);
}
// TESTED
vector<ll> getCatalanNumbers(ll n) {
  vector<ll> C(n + 1, 0);
  C[0] = C[1] = 1;
  C[2] = 2;
  for (ll i = 3; i <= n; ++i) {
    for (ll j = 0; j < i; ++j) C[i] += C[j] * C[i - j - 1];
  }
  return C;
}
```

Convex Hull

```
// TESTED
// Monotic Change
struct point {
  double x, y;
};
ll orientation(const point& a, const point& b, const point& c) {
  double sign = a.x * (b.y - c.y) - b.x * (a.y - c.y) + c.x * (a.y - b.y);
  if (sign > 0)
    return 1;
  else if (sign < 0)
    return -1;
  else
    return 0;
}
bool cw(const point& a, const point& b, const point& c,
        const bool& include_colider) {
  ll o = orientation(a, b, c);
  return (o > 0 || include_colider && 0 == 0);
}
bool ccw(const point& a, const point& b, const point& c,
         const bool& include_colider) {
  ll o = orientation(a, b, c);
  return (o < 0 || include_colider && o == 0);
}
void convex_hull(vector<point>& a, bool include_colider) {
  if (a.size() == 1) return;
  sort(a.begin(), a.end(), [](const point& p1, const point& p2) {
    return (p1.x == p2.x ? p1.y < p2.y : p1.x < p2.x);
```

Convex Hull (cont.)

```
  });
  point A = a[0], B = a[a.size() - 1];
  vector<point> upper, lower;
  upper.push_back(A);
  lower.push_back(A);
  for (ll i = 1; i < a.size(); ++i) {
    if (i == a.size() - 1 || cw(A, a[i], B, include_colider)) {
      while (upper.size() >= 2 &&
             !(cw(upper[upper.size() - 2], upper[upper.size() - 1], a[i],
                  include_colider)))
        upper.pop_back();
      upper.push_back(a[i]);
    }
    if (i == a.size() - 1 || ccw(A, a[i], B, include_colider)) {
      while (lower.size() >= 2 &&
             !(ccw(lower[lower.size() - 2], lower[lower.size() - 1], a[i],
                   include_colider)))
        lower.pop_back();
      lower.push_back(a[i]);
    }
  }
  if (include_colider && upper.size() == a.size()) {
    reverse(a.begin(), a.end());
    return;
  }
  a.clear();
  for (ll i = 0; i < upper.size(); ++i) a.push_back(upper[i]);
  for (ll i = lower.size() - 2; i >= 1; --i) a.push_back(lower[i]);
}
```

Crt

```
// TESTED (PD: for 2x2 system but should work for any nXn)
#include "gcdExtended.cpp"
ll mod_inv(ll a, ll m) {
  ll x, y;
  ll g = gcdExtended(a, m, x, y);
  x = (x % m + m) % m;
  return x;
}
struct Congruence {
  ll b, n;
};
ll chinese_remainder_theorem(vector<Congruence> const& congruences) {
  ll N = 1;
  for (auto const& congruence : congruences) N *= congruence.n;
  ll x = 0;
  for (auto const& congruence : congruences) {
    ll b_i = congruence.b;
    ll N_i = N / congruence.n;
    ll invN_i = mod_inv(N_i, congruence.n);
    i128 aux = (b_i * N_i) % N;
    aux = (aux * invN_i) % N;
    x = (ll)(x + aux) % N;
  }
  return x;
}
```

Discrete Log

```
// Tested
ll discrete_log(ll a, ll b, ll m) {
  a %= m, b %= m;
  if (a == 0) return b == 0 ? 1 : -1;
  ll k = 1, add = 0, g;
  while ((g = gcd(a, m)) > 1) {
    if (b == k) return add;
    if (b % g) return -1;
    b /= g, m /= g, ++add;
    k = (k * 1ll * a / g) % m;
  }
  ll n = sqrt(m) + 1;
  ll an = 1;
  for_(i, n) an = (an * 1ll * a) % m;
  unordered_map<ll, ll> f2inv;
  for (ll q = 0, f2 = b; q <= n; ++q) {
    f2inv[f2] = q;
    f2 = (f2 * 1ll * a) % m;
  }
  for (ll p = 1, f1 = k; p <= n; ++p) {
    f1 = (f1 * 1ll * an) % m;
    if (f2inv.count(f1)) {
      ll x = n * p - f2inv[f1] + add;
      return x;
    }
  }
  return -1;
}
```

Discrete Root

```
// TESTED
#include "bin_pow.cpp"
#include "gcd.cpp"
#include "primitive_root.cpp"
#include "tot_euler.cpp"
vector<ll> DiscreteRoot(ll n, ll k, ll a) {
  if (a == 0) return {0};
  ll g = generator_(n);
  ll sq = (ll)sqrt(n) + 1;
  vector<pair<ll, ll>> dec(sq);
  for (ll i = 1; i <= sq; ++i)
    dec[i - 1] = {binpow(g, i * sq * k % (n - 1), n), i};
  sort(dec.begin(), dec.end());
  ll h0 = -1;
  for_(i, sq) {
    ll my = binpow(g, i * k % (n - 1), n) * a % n;
    auto it = lower_bound(dec.begin(), dec.end(), make_pair(my, 0));
    if (it != dec.end() && it->first == my) {
      h0 = it->second * sq - i;
      break;
    }
  }
  if (h0 == -1) return {-1};
  ll delta = (n - 1) / gcd(k, n - 1);
  vector<ll> x;
  for (ll h = h0 % delta; h < n - 1; h += delta) x.push_back(binpow(g, h, n));
  sort(x.begin(), x.end());
  return x;
}
```

Erathostenes Sieve

```
// Tested
vector<bool> erathostenesSieve(ll n) {
  vector<bool> isprime(n + 1, true);
  vector<ll> primes;
  isprime[0] = isprime[1] = false;
  for (ll i = 2; i <= n; (i == 2 ? i++ : i += 2)) {
    if (isprime[i]) {
      for (ll j = i * i; j <= n; j += i) isprime[j] = false;
      primes.push_back(i);
    }
  }
  return isprime;
}
```

Factorization Pollard Rho

```
// TESTED up to 1e18, run T(n) < 1e8
ll mult(ll a, ll b, ll m) { return (__int128)a * b % m; }
ll f(ll x, ll c, ll m) { return (mult(x, x, m) + c) % m; }
#include "gcd.cpp"
ll rho(ll n, ll x0 = 2, ll c = 1) {
  ll x = x0;
  ll y = x0;
  ll g = 1;
  while (g == 1) {
    x = f(x, c, n);
    y = f(y, c, n);
    y = f(y, c, n);
    g = gcd(abs(x - y), n);
  }
  return g;
}
// algorithm for the factorization
ll brent(ll n, ll x0 = 2, ll c = 1) {
  ll x = x0;
  ll g = 1;
  ll xs, y;
  ll m = 128;
  ll l = 1;
  ll iter = 0;
  while (g == 1 && iter < 1000) {
    y = x;
    ll q = 1;
    for (ll i = 1; i < l; i++) x = f(x, c, n);
    ll k = 0;
    while (k < l && g == 1) {
      xs = x;
      for (ll i = 0; i < m && i < l - k; i++) {
        x = f(x, c, n);
        q = mult(q, abs(y - x), n);
      }
      g = gcd(q, n);
      k += m;
    }
```

Factorization Pollard Rho (cont.)

```
    l *= 2;
    iter++;
  }
  if (g == n) {
    return n;
  }
  return g;
}
#include "miller_rabin.cpp"
// algorithm to prime factorization
void factorize(ll n, map<ll, ll>& factors) {
  if (n == 1) return;
  if (deterministicMillerRabin(n)) {
    factors[n]++;
    return;
  }
  ll d1, d2, iter = 0;
  while (iter < 1000) {
    uniform_int_distribution<ll> dist(1, n - 1);
    ll c = dist(rng);
    ll x0 = dist(rng);
    d1 = brent(n, x0, c), d2 = n / d1;
    if (d1 > 1 && d1 < n) break;
    iter++;
  }
  if (d1 == 1 || d2 == 1) {
    factors[n]++;
    return;
  }
  factorize(d1, factors);
  factorize(d2, factors);
}
```

Fft

```
// Tested
void fft_bottom_up(vector<cd>& c, bool inverse) {
  ll N = 1;
  while (N < c.size()) N <<= 1;
  c.resize(N);
  if (N <= 1) return;
  ll cnt = 0;
  for (ll j = 1; j < N; ++j) {
    ll bit = N >> 1;
    while (cnt & bit) {
      cnt ^= bit;
      bit >>= 1;
    }
    cnt ^= bit;
    if (cnt < j) swap(c[cnt], c[j]);
  }
  // n is the length of the current block of the tree, thanks to bit reversal is
  // in order, n=2 -> depth= D-1, n=4 -> depth=D-2, and so on...
  for (ll n = 2; n <= N; n <<= 1) {
    double ang = 2 * PI / n * (inverse ? -1 : 1);
    cd wn(cos(ang), sin(ang));
    // iterate each block(node from the current depth)
    for (ll currblock = 0; currblock < N; currblock += n) {
      cd w(1);
      // iterate the node of the current block, only recover a half, because
      // butterfly effect(como se diga la wea).
      for (ll curr = 0; curr < n / 2; ++curr) {
        cd u = c[currblock + curr], v = c[currblock + curr + n / 2] * w;
        c[currblock + curr] = u + v;
        c[currblock + curr + n / 2] = u - v;
        w *= wn;
      }
    }
  }
  if (inverse) {
    for (cd& x : c) x /= N;
  }
}
vector<cd> conv(vector<cd> c1, vector<cd> c2) {
  vector<cd> ans;
  ll N = 1;
  while (N < (c1.size() + c2.size() - 1ll)) N <<= 1;
  c1.resize(N);
  c2.resize(N);
  fft_bottom_up(c1, false);
  fft_bottom_up(c2, false);
  ans.resize(N);
  for_(i, N) ans[i] = c1[i] * c2[i];
  fft_bottom_up(ans, true);
  return ans;
}
```

Gaussian Elimination

```
// Tested
bool gauss_jordan(vector<vector<ll>> A, vector<ll>& x) {
  ll n = A.size(), m = A[0].size() - 1;
  vector<ll> where(m, -1);
  for (ll col = 0, row = 0; col < m && row < n; ++col) {
    ll pivot = row;
    for (ll i = row; i < n; ++i) {
      if (abs(A[i][col]) > abs(A[pivot][col])) pivot = i;
    }
    if (!A[pivot][col]) continue;
    for (ll i = col; i <= m; ++i) swap(A[pivot][i], A[row][i]);
    where[col] = row;
    for (ll i = 0; i < n; ++i) {
      ll invm = binpow(A[row][col], MOD - 2, MOD);
      if (i != row) {
        ll c = (A[i][col] * invm) % MOD;
        for (ll j = col; j <= m; ++j)
          A[i][j] = ((A[i][j] - A[row][j] * c) % MOD + MOD) % MOD;
      }
    }
    ++row;
  }
  x.assign(m, 0);
  for (ll i = 0; i < m; ++i) {
    if (where[i] != -1) {
      ll invm = binpow(A[where[i]][i], MOD - 2, MOD);
      x[i] = (A[where[i]][m] * invm) % MOD;
    }
  }
  for (ll i = 0; i < n; ++i) {
    ll sum = 0;
    for (ll j = 0; j < m; ++j) sum += (x[j] * A[i][j]) % MOD;
    if (abs(sum - A[i][m]) > 0) return false;  // No solution
  }
  return true;  // Valid solution, trivial or infinity
```

Gcd

```
// Tested
ll gcd(ll a, ll b) {
  if (b == 0) return a;
  return gcd(b, a % b);
}
ll gcd(ll a, ll b) {
  while (b) {
    ll r = a % b;
    a = b;
    b = r;
  }
  return a;
}
```

Gcd Extended

```
// Tested
ll gcdExtended(ll a, ll b, ll& x, ll& y) {
  if (b == 0) {
    x = 1;
    y = 0;
    return a;
  }
  ll x_, y_, gcd = gcdExtended(b, a % b, x_, y_);
  x = y_;
  y = x_ - y_ * (a / b);
  return gcd;
}
```

Inclusion Exclusion

```
// Example cardinality of all subsets of size 1
ll cardinalitySubset(ll subset) { return 1; }
// Template
ll InclusionExclusion() {
  // sets
  ll n = 10;               // example
  ll subsets = (1 << n);  // total subsets
  ll ans = 0;
  // travel on all the subsets, with a particular subset
  for (ll subset = 0; subset < subsets; subset++) {
    // if is odd the subset we sum, otherwise substract, recall, Union of sets =
    // sum sets - intersect sets pairs + intersect group of three - ...
    ans += (__builtin_popcount(subset) & 1 ? cardinalitySubset(subset)
                                           : -cardinalitySubset(subset));
  }
  return ans;
}
```

Miller Rabin

```cpp
// Tested
ll binpow(ll a, ll b, ll mod) {
  ll ans = 1;
  while (b > 0) {
    if (b & 1) ans = (u128)(ans)*a % mod;
    a = (u128)a * a % mod;
    b >>= 1;
  }
  return ans;
}
bool checkComposite(ll p, ll a, ll s, ll q) {
  ll x = binpow(a, q, p);
  if (x == 1 || x == p - 1) return false;
  for (ll i = 1; i < s; ++i) {
    x = (u128)x * x % p;
    if (x == p - 1) return false;
  }
  return true;
}
bool millerRabin(ll p, ll maxIter = 5) {
  if (p < 4) return p == 2 || p == 3;
  ll s = 0, q = p - 1;
  while (!(q & 1)) {
    s++;
    q >>= 1;
  }
  for (ll i = 0; i < maxIter; ++i) {
    ll a = 2 + rand() % (p - 3);
    if (checkComposite(p, a, s, q)) return false;
  }
  return true;
}
bool deterministicMillerRabin(ll p) {
  if (p < 4) return p == 2 || p == 3;
  ll s = 0, q = p - 1;
  while (!(q & 1)) {
    s++;
    q >>= 1;
  }
  for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    if (p == a) return true;
    if (checkComposite(p, a, s, q)) return false;
  }
  return true;
}
```

Mobius

```cpp
// Tested
vector<ll> mobius(ll n) {
  vector<ll> mob(n + 1, 1);
  mob[1] = 1;
  vector<bool> isprime(n + 1, true);
  isprime[0] = isprime[1] = false;
  for (ll i = 2; i < n + 1; i++) {
    if (isprime[i]) {
      for (ll j = i; j < n + 1; j += i) {
        mob[j] *= -1;
        isprime[j] = false;
      }
      for (ll j = i * i; j < n + 1; j += i * i) mob[j] = 0;
    }
  }
  return mob;
}
```

Ntt

```cpp
// TESTED
const ll mod = 998244353, generator = 3, rootn = 3; // for N max = 2^23
/*more modelus primes of the form p = c*2^m + 1 (generators calculated with algorithm in math section):
const ll mod = 7340033, generator = 3, rootn = 3; // for N max = 2^20
const ll mod = 167772161, generator = 3, rootn = 3; // for N max = 2^25
const ll mod = 469762049, generator = 3, rootn = 3; // for N max = 2^26
*/
#include "bin_pow.cpp"
ll inv(ll a) { return binpow(a, mod - 2); }
void ntt(vector<ll>& c, bool invert) {
  ll N = 1;
  while (N < c.size()) N <<= 1;
  c.resize(N);
  ll cnt = 0;
  for (ll j = 1; j < N; ++j) {
    ll bit = N >> 1;
    while (bit & cnt) {
      cnt ^= bit;
      bit >>= 1;
    }
    cnt ^= bit;
    if (cnt < j) swap(c[cnt], c[j]);
  }
  for (ll n = 2; n <= N; n <<= 1) {
    ll wn = binpow(rootn, (mod - 1) / n);
    if (invert) wn = inv(wn);
    for (ll currblock = 0; currblock < N; currblock += n) {
      ll w = 1;
      for (ll i = 0; i < n / 2; ++i) {
        ll u = c[currblock + i] % mod, v = (w * c[currblock + i + n / 2]) % mod;
        c[currblock + i] = (u + v) % mod;
        c[currblock + i + n / 2] = ((u - v) % mod + mod) % mod;
        w = (w * wn) % mod;
      }
    }
  }
  if (invert) {
    for (ll& x : c) x = (x * inv(N)) % mod;
  }
}
vector<ll> conv(vector<ll> c1, vector<ll> c2) {
  ll N = 1;
  while (N <= (c1.size() + c2.size() - 1ll)) N <<= 1;
  c1.resize(N);
  c2.resize(N);
  ntt(c1, false);
  ntt(c2, false);
  vector<ll> ans(N, 0);
  for_(i, N) ans[i] = (c1[i] * c2[i]) % mod;
  ntt(ans, true);
  return ans;
}
```

Primitive Root

```cpp
// TESTED
#include "bin_pow.cpp"
#include "tot_euler.cpp"
ll generator_(ll n) {
  vector<int> facts;
  ll phi_n = tot_euler(n), m = phi_n;
  for (ll i = 2; i * i <= m; ++i) {
    if (m % i == 0) {
      facts.push_back(i);
      while (m % i == 0) m /= i;
    }
  }
  if (m > 1) facts.push_back(m);
  for (ll g = 2; g <= n; ++g) {
    bool ok = true;
    for (size_t i = 0; i < facts.size() && ok; ++i)
      ok &= binpow(g, phi_n / facts[i], n) != 1;
    if (ok) return g;
  }
  return -1;
}
```

Simpson Rule

```cpp
// TESTED
ld f(ld x) { return x; }
ld simpson_integration(ld a, ld b, ld d) {
  const ll N = 10000;  // number of steps (already multiplied by 2)
  ld h = (b - a) / N;
  ld s = f(a) + f(b);            // a = x_0 and b = x_2n
  for (ll i = 1; i < N; i++) {  // Refer to final Simpson's formula
    double xi = a + h * i;
    s += f(xi) * ((i & 1) ? 4 : 2);
  }
  s *= h / 3.0;
  // 1e6 is the decimal precision, if you want 7, the 1e7 and so on
  s *= 1e6;
  s = llround(s);
  s /= 1e6;
  // to print use cout << fixed << setprecision(6) << s;
  return s;
}
```

Template

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ll long long
#define ld long double
#define for_(i, n) for (ll i = 0; i < n; ++i)
#define ull unsigned long long
#define INF LLONG_MAX
#define NINF LLONG_MIN
#define i128 __int128
```

Template (cont.)

```cpp
#define u128 __uint128_t
#define cd complex<ld>
const double PI = 3.14159265358979323846264338327950288419716939937510;
const double EPS = 1e-18;
```

Tonelli Shanks

```cpp
// TESTED
#include "bin_pow.cpp"
ll TonelliShanks(ll n, ll p) {
  n %= p;
  if (n < 0) n += p;
  if (n == 0) return 0;
  if (binpow(n, (p - 1) / 2, p) != 1) return -1;  // solution does not exist
  if (p % 4 == 3)
    return binpow(n, (p + 1) / 4, p);  // trivial solution, only one factor 2
  ll c = p - 1, z = 2;
  ll s = 0;
  while (c % 2 == 0) ++s, c /= 2;
  // find a non-square mod p
  while (binpow(z, (p - 1) / 2, p) != p - 1) ++z;
  ll q = binpow(z, c, p);            // generator subgroup, |<q>| = 2^s
  ll a = binpow(n, (c + 1) / 2, p);  // candidate root
  ll t =
      binpow(n, c, p);  // error from equation a^2 = n*t, if t=1, a is the root
  ll m = s;
  while (1) {
    ll i = 0, temp = t;
    while (i < m && temp != 1) {
      temp = (temp * temp) % p;
      i++;
    }
    if (i == 0) return a;
    ll exponent = 1ll << (m - i - 1);
    ll b = binpow(q, exponent, p);
    a = (a * b) % p;
    t = (((t * b) % p) * b) % p;
    q = (b * b) % p;
    m = i;
  }
}
```

Tot Euler

```cpp
// Tested
// sieve
vector<ll> tot_euler(ll n) {
  vector<ll> tot(n + 1, 0);
  for_(i, n + 1) tot[i] = i;
  for (ll i = 2; i <= n; ++i) {
    if (tot[i] == i) {
      for (ll j = i; j <= n; j += i) tot[j] -= tot[j] / i;
    }
  }
  return tot;
}
// for a simply n
ll tot_euler(ll n) {
  ll n_ = n, tot = n;
  for (ll i = 2; i * i <= n; i++) {
    if (n_ % i == 0) {
      tot -= tot / i;
      while (n_ % i == 0) n_ /= i;
    }
  }
  if (n_ > 1) tot -= tot / n_;
  return tot;
}
```

Z Transform

```cpp
// TESTED for gcd
// only if it is neccesary, but Z transform work on any module, and without
ll mod = 998244353;
void Z(vector<ll>& C, ll N, bool g) {
  if (g) {
    for (ll d = 1; d <= N; d++) {
      for (ll m = 2 * d; m <= N; m += d) C[d] = (C[d] + C[m]) % mod;
    }
  } else {
    for (ll d = N; d > 0; d--) {
      for (ll m = 2 * d; m <= N; m += d) C[m] = (C[m] + C[d]) % mod;
    }
  }
}
void Zinv(vector<ll>& C, ll N, bool g) {
```

Z Transform (cont.)

```cpp
  if (g) {
    for (ll d = N; d > 0; d--) {
      for (ll m = 2 * d; m <= N; m += d)
        C[d] = (mod + (C[d] - C[m]) % mod) % mod;
    }
  } else {
    for (ll d = 1; d <= N; d++) {
      for (ll m = 2 * d; m <= N; m += d)
        C[m] = (mod + (C[m] - C[d]) % mod) % mod;
    }
  }
}
vector<ll> conv(vector<ll> A, vector<ll> B, bool g) {
  ll N = A.size() - 1ll;
  Z(A, N, g);
  Z(B, N, g);
  vector<ll> C(N + 1ll);
  for (ll i = 1; i <= N; i++) C[i] = (A[i] * B[i]) % mod;
  Zinv(C, N, g);
  return C;
}
```

Strings

Aho Corasick

```cpp
#define sz(x) ((int)(x).size())
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
typedef vector<int> vi;
struct AhoCorasick {
  enum { alpha = 26, first = 'a' };  // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) {
        n = m = sz(N);
        N.emplace_back(-1);
      } else
        n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i, 0, sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i, 0, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1)
          ed = y;
        else {
          N[ed].back = y;
          if (N[ed].end == -1) {
            N[ed].end = N[y].end;
          } else {
            backp[N[ed].start] = N[y].end;
          }
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res;  // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
```

Aho Corasick (cont.)

```
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

Booth

```
int Booth(string s) {
  s += s;
  vector<int> t(s.size(), -1);
  int ans = 0;
  int n = s.size();
  for (int i = 1; i < n; i++) {
    char c = s[i];
    int j = t[i - ans - 1];
    while (j != -1 && c != s[ans + j + 1]) {
      if (c < s[ans + j + 1]) {
        ans = i - j - 1;
      }
      j = t[j];
    }
    if (c != s[ans + 1 + j]) {
      if (c < s[ans]) {
        ans = i;
      }
      t[i - ans] = -1;
    } else {
      t[i - ans] = j + 1;
    }
  }
  return ans;
}
```

Kmp

```
struct KMP {
  vector<int> lps;
  string s;
  KMP(string s) : s(s) {
    int n = s.size();
    lps.assign(n, 0);
    int len = 0, i = 1;
    while (i < n) {
      if (s[i] == s[len]) {
        lps[i++] = ++len;
      } else {
        if (len != 0)
          len = lps[len - 1];
        else
          lps[i++] = 0;
      }
    }
  }
};
```

Manacher

```
vector<int> manacher_odd(string s) {
  int n = s.size();
  s = "$" + s + "^";
  vector<int> p(n + 2);
  int l = 0, r = 1;
  for (int i = 1; i <= n; i++) {
    if (i <= r) {
      p[i] = min(r - i, p[l + (r - i)]);
    }
    while (s[i - p[i]] == s[i + p[i]]) {
      p[i]++;
    }
    if (i + p[i] > r) {
      l = i - p[i], r = i + p[i];
    }
  }
  return vector<int>(begin(p) + 1, end(p) - 1);
}

vector<int> manacher(string s) {
  string t;
  for (auto c : s) {
    t += string("#") + c;
  }
  auto res = manacher_odd(t + "#");
  /*
  int ans_largo = v[i]-1;
  if(i%2 == 0){
      int indice = i/2;
      int l = indice - (ans_largo/2);
      int r = indice + (ans_largo/2);
  } else {
```

Manacher (cont.)

```
      int indice = (i+1)/2;
      int l = indice - (ans_largo/2);
      int r = indice + (ans_largo/2)-1;
  }
  palindromo -> s[l , r] <-- r incluido
  */
  return vector<int>(begin(res) + 1, end(res) - 1);
}
```

Rolling Hashing

```
template <class T>
struct rolling_hashing {
  int base, mod;
  vector<long long> p, H;
  int n;
  rolling_hashing(const T& s, int b, int m) : base(b), mod(
m), n(s.size()) {
    p.assign(n + 1, 1);
    H.assign(n + 1, 0);
    for (int i = 0; i < n; ++i) {
      H[i + 1] = (H[i] * base + s[i]) % mod;
      p[i + 1] = (p[i] * base) % mod;
    }
  }
  int get(int l, int r) {
    int res = (H[r + 1] - H[l] * p[r - l + 1]) % mod;
    if (res < 0) res += mod;
    return res;
  }
};
```

Suffix Array

```
struct SA {
  int n;
  vector<int> C, R, R_, sa, sa_, lcp;
  inline int gr(int i) { return i < n ? R[i] : 0; }
  void csort(int maxv, int k) {
    C.assign(maxv + 1, 0);
    for (int i = 0; i < n; i++) C[gr(i + k)]++;
    for (int i = 1; i < maxv + 1; i++) C[i] += C[i - 1];
    for (int i = n - 1; i >= 0; i--) sa_[--C[gr(sa[i] + k)]
] = sa[i];
    sa.swap(sa_);
  }
  void getSA(vector<int>& s) {
    R = R_ = sa = sa_ = vector<int>(n);
    for (ll i = 0; i < n; i++) sa[i] = i;
    sort(sa.begin(), sa.end(), [&s](int i, int j) { return
s[i] < s[j]; });
    int r = R[sa[0]] = 1;
    for (ll i = 1; i < n; i++) R[sa[i]] = (s[sa[i]] != s[sa
[i - 1]]) ? ++r : r;
    for (int h = 1; h < n && r < n; h <<= 1) {
      csort(r, h);
      csort(r, 0);
      r = R_[sa[0]] = 1;
      for (int i = 1; i < n; i++) {
        if (R[sa[i]] != R[sa[i - 1]] || gr(sa[i] + h) != gr
(sa[i - 1] + h)) r++;
        R_[sa[i]] = r;
      }
      R.swap(R_);
    }
  }
  void getLCP(vector<int>& s) {
    lcp.assign(n, 0);
    int k = 0;
    for (ll i = 0; i < n; i++) {
      int r = R[i] - 1;
      if (r == n - 1) {
        k = 0;
        continue;
      }
      int j = sa[r + 1];
      while (i + k < n && j + k < n && s[i + k] == s[j + k]
) k++;
      lcp[r] = k;
      if (k) k--;
    }
  }
  SA(vector<int>& s) {
    n = s.size();
    getSA(s);
    getLCP(s);
  }
};
```

Trie

```
struct trie {
  vector<vector<ll>> tree;
  vector<ll> counts;
```

Trie (cont.)

```
  trie() {
    tree.push_back(vector<ll>(26, -1));
    counts.push_back(0);
  };
  void insert(string& s) {
    ll i = 0, u = 0;
    while (i < s.size()) {
      char c = s[i];
      if (tree[u][c - 'a'] == -1) {
        ll pos = tree.size();
        tree.push_back(vector<ll>(26, -1));
        counts.push_back(0);
        tree[u][c - 'a'] = pos;
      }
      i++;
      u = tree[u][c - 'a'];
    }
    counts[u]++;  // count final character
  }
  ll search(string& s) {
    ll i = 0, u = 0;
    ll count_ends = 0;
    while (i < s.size()) {
      char c = s[i];
      if (tree[u][c - 'a'] != -1) {
        // If you need to know how many trailing characters it goes through,
        // uncomment. count_ends += counts[u];
        u = tree[u][c - 'a'];
        i++;
      } else
        break;
    }
    if (i == s.size()) count_ends += counts[u];
    return count_ends;
  }
};
```

Zfunction

```
struct Z {
  string s;
  vector<int> z;
  ll n;
  Z(string s) : s(s) {
    n = s.size();
    z.assign(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
      if (i < r) {
        z[i] = min(r - i, z[i - l]);
      }
      while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
        z[i]++;
      }
      if (i + z[i] > r) {
        l = i;
        r = i + z[i];
      }
    }
  }
};
```

Programacion Dinamica

Knappsack

```
void solve() {
  ll n, w;
  cin >> n >> w;
  vector<pair<ll, ll>> v(n);
  for (auto& [a, b] : v) cin >> a >> b;
  vector<ll> ans(w + 2, 0);
  ll ans_ = 0;
  for (auto [vi, wi] : v) {
    for (ll j = w; j >= wi; j--) {
      ans[j] = max(ans[j], ans[j - wi] + vi);
      ans_ = max(ans_, ans[j]);
    }
  }
  cout << ans_ << endl;
}
```

Lis

```
template <class I>
vector<int> LIS(const vector<I>& S) {
  if (S.empty()) return {};
  vector<int> prev(S.size());
  vector<pair<I, int>> res;
  for (int i = 0; i < S.size(); i++) {
    auto it = lower_bound(res.begin(), res.end(), pair<I, int>{S[i], i});
    if (it == res.end()) res.emplace_back(), it = res.end() - 1;
    *it = {S[i], i};
    prev[i] = (it == res.begin() ? 0 : (it - 1)->second);
  }
  int L = res.size(), cur = res.back().second;
  vector<int> ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  /* Recover the sequence
  for (int i = 0; i+1 < ans.size(); i++)
    ans[i] = S[ans[i]];
  */
  return ans;
}
```

Lis Length K

```
vector<fenwick_tree> bit(k + 1, fenwick_tree(m));

ll answer = 0;
for (ll x : v) {
  ll value = compress(x);  // or already compressed
  vector<ll> current(k + 1);
  current[1] = 1;
  for (ll len = 2; len <= k; len++) {
    current[len] = bit[len - 1].sum(value - 1);
  }
  for (ll len = 1; len <= k; len++) {
    bit[len].add(value, current[len]);
  }
  answer += current[k];
}
cout << answer << endl;
```

Offline Algorithms

Cdq

```
void cdq(ll l, ll r, vector<pt>& v) {
  if (r - l <= 1) return;
  ll m = (l + r) / 2;
  cdq(l, m, v);
  cdq(m, r, v);
  vector<pt> tmp;
  ll a = l, b = m;
  while (a < m && b < r) {
    if (v[a].y < v[b].y) {
      st.add(v[a].z, 1);
      tmp.push_back(v[a++]);
    } else {
      ans[v[b].id] += st.sum(v[b].z - 1);
      tmp.push_back(v[b++]);
    }
  }
  ll record = a;
  while (a < m) tmp.push_back(v[a++]);
  while (b < r) {
    ans[v[b].id] += st.sum(v[b].z - 1);
    tmp.push_back(v[b++]);
  }
  for (ll i = l; i < record; i++) st.add(v[i].z, -1);
  for (ll i = l; i < r; i++) v[i] = tmp[i - l];
}
/*
sort(v, v + n, [](const pt &a, const pt &b) {
    if (a.x != b.x) return a.x < b.x;
    if (a.y != b.y) return a.y > b.y; // Opposite of merge condition!
    return a.z > b.z;
});
*/
```

Mo

```
struct query {
  ll l, r, i;
};
template <class T, typename U, class T2>
struct mo_algorithm {
  vector<U> ans;
  template <typename... Args>
  mo_algorithm(vector<T>& v, vector<query>& queries, Args... args) {
    T2 ds(args...);
    ll block_sz = sqrtl(v.size());
    ans.assign(queries.size(), -1);
    sort(queries.begin(), queries.end(), [&](auto& a, auto& b) {
      return a.l / block_sz != b.l / block_sz ? a.l / block_sz < b.l / block_sz
                                              : a.r < b.r;
    });
    ll l = 0, r = -1;
    for (query q : queries) {
      while (l > q.l) ds.add(v[--l]);
```

Mo (cont.)

```
      while (r < q.r) ds.add(v[++r]);
      while (l < q.l) ds.remove(v[l++]);
      while (r > q.r) ds.remove(v[r--]);
      ans[q.i] = ds.answer(q);
    }
  }
};
```

Pbs

```
vector<ll> l(q, 0), r(q, m + 1);
while (1) {
  vector<vector<ll>> bucket(m + 1);
  bool flag = 0;
  for (ll i = 0; i < q; i++) {
    if (r[i] - l[i] <= 1) continue;
    flag = 1;
    bucket[(l[i] + r[i]) >> 1].push_back(i);
  }
  if (!flag) break;
  // RESET DATA STRUCTURE

  // SWEEP THROUGH POSIBLE ANSWERS POSITIONS
  for (ll i = 1; i <= m; i++) {
    update(i);

    for (auto j : bucket[i]) {
      if (foo(j))
        r[j] = i;
      else
        l[j] = i;
    }
  }
}
```

Value Driven

```
struct event {
  ll l, r, x;  // For update value, for query threshold
  ll id, q;    // query type 0 update
};
vector<ll> ans;
void value_driven(ll l, ll r, vector<event>& v) {
  if (v.empty()) return;
  if (r - l <= 1) {
    // COUNTING -> return;
    for (auto& i : v) {
      if (i.q == 1) ans[i.id] = l;
    }
    return;
  }
  ll m = (l + r) >> 1;
  vector<event> vl, vr;
  for (auto& i : v) {
    if (i.q == 0) {
      if (i.x >= m) {
        // UPDATE
        // st.add(i.l, 1);
        vr.push_back(i);
      } else {
        vl.push_back(i);
      }
    } else {
      // ll cnt = query;
      // ll cnt = st.sum(i.l, i.r);
      if (cnt >= i.x) {
        vr.push_back(i);
      } else {
        // COUNTING -> ans[i.id] += cnt; push_back(i);
        event ni = i;
        ni.x -= cnt;
        vl.push_back(ni);
      }
    }
  }
  // ROLLBACK
  // for (auto &i: v) {
  //     if (i.q == 0 && i.x >= m) {
  //         st.add(i.l, -1);
  //     }
  // }
  value_driven(l, m, vl);
  value_driven(m, r, vr);
}
```

Miscelaneo

Busqueda Binaria

```
int bs(vector<ll>& v, int b) {
  int l = 0, r = v.size();
  while (l < r) {
    int mid = (l + r) / 2;
    if (v[mid] >= b)
```

Busqueda Binaria (cont.)

```
      r = mid;
    else
      l = mid + 1;
  }
  return l;
}
```

Golden Search

```
double golden_search(double l, double r) {
  double phi = (1 + sqrt(5)) / 2;
  double eps = 1e-9;
  while (r - l > eps) {
    double m1 = r - (r - l) / phi;
    double m2 = l + (r - l) / phi;
    if (f(m1) < f(m2)) l = m1;
    else r = m2;
  }
  return l;
}
```

Random

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());

// Número aleatorio entero [a, b]
int random(int a, int b) {
    return uniform_int_distribution<int>(a, b)(rng);
}
// Número aleatorio long long [a, b]
ll random(ll a, ll b) {
    return uniform_int_distribution<ll>(a, b)(rng64);
}
// shuffle a un vector v
shuffle(v.begin(), v.end(), rng);

```

Time

```
using Clock = chrono::steady_clock;
using TimePoint = Clock::time_point;
// Obtener instante actual
TimePoint getTime() { return Clock::now(); }

// Diferencia en segundos
double elapsedSeconds(TimePoint start) {
  return chrono::duration<double>(Clock::now() - start).count();
}

// Diferencia en milisegundos
ll elapsedMilliseconds(TimePoint start) {
  return chrono::duration_cast<chrono::milliseconds>(Clock::now() - start)
      .count();
}

// Diferencia en microsegundos
ll elapsedMicroseconds(TimePoint start) {
  return chrono::duration_cast<chrono::microseconds>(Clock::now() - start)
      .count();
}

// Diferencia en nanosegundos
ll elapsedNanoseconds(TimePoint start) {
  return chrono::duration_cast<chrono::nanoseconds>(Clock::now() - start)
      s.count();
}

// Dormir durante cierta cantidad de tiempo
void sleepSeconds(int s) { this_thread::sleep_for(chrono::seconds(s)); }

void sleepMilliseconds(int ms) {
  this_thread::sleep_for(chrono::milliseconds(ms));
}
```

Xorshift

```
uint32_t xorshift() {
    static uint32_t seed = static_cast<uint32_t>(time(nullptr));
    seed ^= seed << 13;
    seed ^= seed >> 17;
    seed ^= seed << 5;
    return seed;
}
```

Geometria

Circle-intersection

```cpp
/*
 *Description:* Computes the pair of points where two circles intersect. Returns
 * false if no intersection. Uses point2d. Status:* KACTL based, not self tested
 */
using P = point2d<ld>;
bool circle_inter(P a, P b, ld r1, ld r2, pair<P, P>* out) {
  if (a.sqdist(b) < 1e-18) return false;
  P vec = b - a;
  ld d2 = vec.norm(), sum = r1 + r2, dif = r1 - r2,
     p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2), h2 = r1 * r1 - p * p * d2;
  if (sum * sum < d2 || dif * dif > d2) return false;
  P per = {-vec.y, vec.x};
  P mid = a + vec * p;
  per = per * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

Circle-polygon-intersection

```cpp
/*
 *Description:* Area of intersection of a circle (center $c$, radius $r$) with a
 * CCW polygon. $O(n)$. Uses point2d. Status:* KACTL based, not self tested
 */
using P = point2d<ld>;
ld circle_poly(P c, ld r, vector<P>& ps) {
  auto arg = [](P p, P q) { return atan2(p ^ q, p | q); };
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = (d | p) / d.norm(), b = (p.norm() - r * r) / d.norm();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0.0L, -a - sqrt(det));
    auto t = min(1.0L, -a + sqrt(det));
    if (t < 0 or 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = p + d * t;
    return arg(p, u) * r2 + (u ^ v) / 2 + arg(v, q) * r2;
  };
  ld sum = 0;
  int n = ps.size();
  for (int i = 0; i < n; i++) sum += tri(ps[i] - c, ps[(i + 1) % n] - c);
  return sum;
}
```

Circle-tangents

```cpp
/*
 *Description:* External tangents of two circles (negate $r_2$ for internal).
 * Returns 0, 1 or 2 tangents. `.first`/`.second` = tangency points on circle
 * 1/2. Set $r_2 = 0$ for point tangents. Uses point2d. Status:* KACTL based,
 * not self tested
 */
using P = point2d<ld>;
vector<pair<P, P>> tangents(P c1, ld r1, P c2, ld r2) {
  P d = c2 - c1;
  ld dr = r1 - r2, d2 = d.norm(), h2 = d2 - dr * dr;
  if (d2 < 1e-18 || h2 < 0) return {};
  vector<pair<P, P>> out;
  P perp = {-d.y, d.x};
  for (ld sign : {-1.0, 1.0}) {
    P v = (d * dr + perp * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 < 1e-18) out.pop_back();
  return out;
}
```

Circle

```cpp
/*
 *Description:* 2D circle, constructors: empty, (center, radius), (a, b, c)
 * circumcircle Status:* Tested
 */
template <typename T>
struct circle {
  point2d<T> c;
  T r;
  circle() : c(), r(0) {}
  circle(point2d<T> c_, T r_) : c(c_), r(r_) {}
  circle(point2d<T> a, point2d<T> b, point2d<T> p) {
    point2d<T> ab = b - a, ap = p - a;
    T d = 2 * (ab ^ ap);
    if (sign(d) == 0) {
      c = point2d<T>();
      r = 0;
      return;
    }
    T s = ab.norm(), t = ap.norm();
    c = a + point2d<T>(ap.y * s - ab.y * t, ab.x * t - ap.x * s) / d;
    r = c.sqdist(a);
    r = sqrtl(r);
  }
};
```

Convex-hull-3d

```cpp
/*
 *Description:* 3D Convex Hull using randomized incremental algorithm. Exact integer arithmetic. Sets `degenerate=true` if less than 4 non-coplanar points.
 *Time:* $O(n log n)$ expected
 *Status:* Tested
 */
template <typename T> struct convex_hull_3d {
  using pt = point3d<T>;
  struct edge;
  struct face {
    int a, b, c;
    pt q;
    edge *e1, *e2, *e3;
    vector<int> points;
    int dead = 1e9;
    face(int a, int b, int c, pt pa, pt pb, pt pc)
        : a(a), b(b), c(c) {
      q = (pb - pa) ^ (pc - pa);
      e1 = e2 = e3 = nullptr;
    }
  };
  struct edge { edge *rev; face *f; };

  vector<pt> p;
  vector<face *> f;
  bool degenerate = false;

  static void glue(face *F1, face *F2, edge *&e1, edge *&e2) {
    e1 = new edge; e2 = new edge;
    e1->rev = e2; e2->rev = e1;
    e1->f = F2; e2->f = F1;
  }

  void prepare() {
    int n = (int)p.size();
    mt19937 rng(42);
    shuffle(p.begin(), p.end(), rng);
    vector<int> ve = {0};
    for (int i = 1; i < n && (int)ve.size() < 4; i++) {
      if (ve.size() == 1) {
        if ((p[ve[0]] - p[i]).nonzero()) ve.push_back(i);
      } else if (ve.size() == 2) {
        if (((p[ve[1]] - p[ve[0]]) ^ (p[i] - p[ve[0]])).nonzero())
          ve.push_back(i);
      } else {
        if ((p[i] - p[ve[0]]).dot((p[ve[1]] - p[ve[0]]) ^ (p[ve[2]] - p[ve[0]])) != 0)
          ve.push_back(i);
      }
    }
    if ((int)ve.size() < 4) { degenerate = true; return; }
    vector<pt> ve2;
    for (int i : ve) ve2.push_back(p[i]);
    reverse(ve.begin(), ve.end());
    for (int i : ve) p.erase(p.begin() + i);
    p.insert(p.begin(), ve2.begin(), ve2.end());
  }

  convex_hull_3d() {}
  convex_hull_3d(vector<pt> points) : p(move(points)) { build_hull(); }

  void build_hull() {
    int n = (int)p.size();
    prepare();
    if (degenerate) return;
    vector<face *> new_face(n, nullptr);
    vector<vector<face *>> conflict(n);
    auto add_face = [&](int a, int b, int c) {
      face *F = new face(a, b, c, p[a], p[b], p[c]);
      f.push_back(F); return F;
    };
    face *F1 = add_face(0, 1, 2);
    face *F2 = add_face(0, 2, 1);
```

Convex-hull-3d (cont.)

```
    glue(F1, F2, F1->e1, F2->e3);
    glue(F1, F2, F1->e2, F2->e2);
    glue(F1, F2, F1->e3, F2->e1);
    for (int i = 3; i < n; i++) {
      for (face *F : {F1, F2}) {
        T Q = (p[i] - p[F->a]).dot(F->q);
        if (Q > 0) conflict[i].push_back(F);
        if (Q >= 0) F->points.push_back(i);
      }
    }
    for (int i = 3; i < n; i++) {
      for (face *F : conflict[i]) F->dead = min(F->dead, i);
      int v = -1;
      for (face *F : conflict[i]) {
        if (F->dead != i) continue;
        int parr[3] = {F->a, F->b, F->c};
        edge *earr[3] = {F->e1, F->e2, F->e3};
        for (int j = 0; j < 3; j++) {
          int j2 = (j + 1) % 3;
          if (earr[j]->f->dead > i) {
            face *Fn = new_face[parr[j]] = add_face(parr[j], parr[j2], i);
            set_union(F->points.begin(), F->points.end(),
                      earr[j]->f->points.begin(), earr[j]->f->points.end(),
                      back_inserter(Fn->points));
            Fn->points.erase(
                stable_partition(Fn->points.begin(), Fn->points.end(),
                                 [&](int k) { return k > i && (p[k] - p[Fn->a]).dot(Fn->q) > 0; }),
                Fn->points.end());
            for (int k : Fn->points) conflict[k].push_back(Fn);
            earr[j]->rev->f = Fn;
            Fn->e1 = earr[j];
            v = parr[j];
          }
        }
      }
      if (v == -1) continue;
      while (new_face[v]->e2 == nullptr) {
        int u = new_face[v]->b;
        glue(new_face[v], new_face[u], new_face[v]->e2, new_face[u]->e3);
        v = u;
      }
    }
    f.erase(remove_if(f.begin(), f.end(),
                      [&](face *F) { return F->dead < n; }), f.end());
  }

  int num_faces() const { return (int)f.size(); }
  double area() const {
    double res = 0;
    for (face *F : f) res += F->q.abs() / 2.0;
    return res;
  }
  double volume() const {
    double vol = 0;
    for (face *F : f) vol += p[F->a].dot(F->q);
    return std::abs(vol) / 6.0;
  }
  vector<array<int, 3>> faces() const {
    vector<array<int, 3>> res;
    for (face *F : f) res.push_back({F->a, F->b, F->c});
    return res;
  }
};
```

Convex-hull

```
/*
 *Description:* Get the convex hull of the cloud of points, allow collinear points if is needed
 *Status:* Highly tested
*/
template<typename T>
vector<point2d<T>> convex_hull(vector<point2d<T>> cloud, bool ac = 0) {
  int n = cloud.size(), k = 0;
  sort(cloud.begin(), cloud.end(), [](point2d<T> &a, point2d<T> &b) {
    return a.x < b.x or (a.x == b.x and a.y < b.y);
  });
  if (n <= 2) return cloud;
  bool allCollinear = true;
  for (int i = 2; i < n; ++i) {
    if (((cloud[1] - cloud[0]) ^ (cloud[i] - cloud[0])) != 0) {
      allCollinear = false; break;
    }
  }
```

Convex-hull (cont.)

```
  if (allCollinear) return ac ? cloud : vector<point2d<T>>{cloud[0], cloud.back()};
  vector<point2d<T>> ch(2 * n);
  auto process = [&](int st, int end, int stp, int t, auto cmp) {
    for (int i = st; i != end; i += stp) {
      while (k >= t and cmp(ch[k - 1], ch[k - 2], cloud[i])) k--;
      ch[k++] = cloud[i];
    }
  };
  process(0, n, 1, 2, [&](auto a, auto b, auto c) {
    return ((a - b) ^ (c - b)) < (ac ? 0 : 1);
  });
  process(n - 2, -1, -1, k + 1, [&](auto a, auto b, auto c) {
    return ((a - b) ^ (c - b)) < (ac ? 0 : 1);
  });
  ch.resize(k - 1);
  return ch;
}
```

Delaunay-graph

```
/*
 *Description:* Planar graph of a Delaunay Triangulation with greedy walk and locate. `walk` finds nearest site. `locate` finds all equidistant nearest sites via BFS: 1 site $=$ Voronoi region, 2 $=$ edge, 3+ $=$ vertex (cocircular).
 *Time:* Build $O(n log n)$ | Walk $O(sqrt(n))$ expected | Locate $O(sqrt(n))$ expected
 *Status:* Tested
 *Dependencies:* point2d, delaunay
 */
template <typename T> struct delaunay_graph : delaunay<T> {
  vector<vector<int>> adj;
  delaunay_graph() {}
  delaunay_graph(vector<T> &px, vector<T> &py) { build(px, py); }
  void build(vector<T> &px, vector<T> &py) {
    delaunay<T>::build(px, py);
    adj.assign(this->n, {});
    for (auto &t : this->tri)
      for (int i = 0; i < 3; i++) for (int j = i+1; j < 3; j++) {
        adj[t[i]].push_back(t[j]); adj[t[j]].push_back(t[i]);
      }
    for (auto &v : adj) { sort(v.begin(), v.end()); v.erase(unique(v.begin(), v.end()), v.end()); }
  }
  int walk(vector<T> &px, vector<T> &py, T qx, T qy) const {
    auto d2 = [&](int i) { return (qx-px[i])*(qx-px[i]) + (qy-py[i])*(qy-py[i]); };
    int cur = 0; T best = d2(0);
    for (bool ok = true; ok; ) { ok = false;
      for (int nb : adj[cur]) { T d = d2(nb); if (d < best) { best=d; cur=nb; ok=true; break; } }
    }
    return cur;
  }
  vector<int> locate(vector<T> &px, vector<T> &py, T qx, T qy) const {
    auto d2 = [&](int i) { return (qx-px[i])*(qx-px[i]) + (qy-py[i])*(qy-py[i]); };
    int v = walk(px, py, qx, qy);
    T best = d2(v);
    vector<int> res = {v};
    vector<bool> vis(this->n, false);
    vis[v] = true;
    for (int qi = 0; qi < (int)res.size(); qi++)
      for (int nb : adj[res[qi]])
        if (!vis[nb]) { vis[nb] = true; if (d2(nb) == best) res.push_back(nb); }
    sort(res.begin(), res.end());
    return res;
  }
};
```

Delaunay

```
/*
 *Description:* Delaunay Triangulation using divide and conquer with quad-edges. Each circumcircle contains none of the input points. If all points are collinear, `tri` is empty. Uses `__int128` for circumcircle test (coords up to $10^9$).
 *Time:* $O(n log n)$
 *Status:* Tested
 *Dependencies:* point2d
 */
template <typename T> struct delaunay {
  using pt = point2d<T>;
```

Delaunay (cont.)

```
  using lll = __int128_t;
  struct Quad {
    Quad *rot, *o;
    pt p; int id; bool mark;
    pt& F() { return rot->rot->p; }
    int& Fi() { return rot->rot->id; }
    Quad*& r() { return rot->rot; }
    Quad* prev() { return rot->o->rot; }
    Quad* next() { return r()->prev(); }
  };
  Quad* pool;
  bool circ(pt p, pt a, pt b, pt c) {
    lll p2=p.norm(), A=a.norm()-p2, B=b.norm()-p2, C=c.norm()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
  }
  Quad* makeEdge(pt orig, int oi, pt dest, int di) {
    auto nq = [&]() { return new Quad{nullptr,nullptr,{}, -1, false}; };
    Quad* r;
    if (pool) { r = pool; pool = r->o; }
    else { r = nq(); r->rot = nq(); r->rot->rot = nq(); r->rot->rot->rot = nq(); }
    r->r()->r() = r; r->rot->rot->rot->rot = r;
    pt arb{(T)1e18, (T)1e18};
    Quad* c = r;
    for (int i = 0; i < 4; i++) {
      c = c->rot; c->p = arb; c->id = -1;
      c->o = (i & 1) ? c : c->r();
    }
    r->p = orig; r->id = oi; r->F() = dest; r->Fi() = di;
    return r;
  }
  void splice(Quad* a, Quad* b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
  }
  Quad* connect(Quad* a, Quad* b) {
    Quad* q = makeEdge(a->F(), a->Fi(), b->p, b->id);
    splice(q, a->next()); splice(q->r(), b); return q;
  }
  pair<Quad*,Quad*> rec(vector<pt>& s, vector<int>& ids, int lo, int hi) {
    int sz = hi - lo;
    if (sz <= 3) {
      Quad* a = makeEdge(s[lo],ids[lo], s[lo+1],ids[lo+1]);
      Quad* b = makeEdge(s[lo+1],ids[lo+1], s[lo+sz-1],ids[lo+sz-1]);
      if (sz == 2) return {a, a->r()};
      splice(a->r(), b);
      auto side = s[lo].cross(s[lo+1], s[lo+2]);
      Quad* c = side ? connect(b, a) : nullptr;
      return {side<0 ? c->r() : a, side<0 ? c : b->r()};
    }
    int mid = lo + sz/2;
    auto [ra, A] = rec(s, ids, lo, mid);
    auto [B, rb] = rec(s, ids, mid, hi);
    while ((B->p.cross(A->F(), A->p)<0 && (A=A->next())) ||
           (A->p.cross(B->F(), B->p)>0 && (B=B->r()->o)));
    Quad* base = connect(B->r(), A);
    if (A->p.x==ra->p.x && A->p.y==ra->p.y) ra = base->r();
    if (B->p.x==rb->p.x && B->p.y==rb->p.y) rb = base;
    auto valid = [&](Quad* e) { return e->F().cross(base->F(), base->p) > 0; };
    for (;;) {
      Quad* LC = base->r()->o;
      if (valid(LC)) while (circ(LC->o->F(), base->F(), base->p, LC->F()))
        { Quad* t=LC->o; splice(LC,LC->prev()); splice(LC->r(),LC->r()->prev()); LC->o=pool; pool=LC; LC=t; }
      Quad* RC = base->prev();
      if (valid(RC)) while (circ(RC->prev()->F(), base->F(), base->p, RC->F()))
        { Quad* t=RC->prev(); splice(RC,RC->prev()); splice(RC->r(),RC->r()->prev()); RC->o=pool; pool=RC; RC=t; }
      if (!valid(LC) && !valid(RC)) break;
      if (!valid(LC) || (valid(RC) && circ(RC->F(), base->F(), base->p, LC->F())))
        base = connect(RC, base->r());
      else base = connect(base->r(), LC->r());
    }
    return {ra, rb};
  }
  int n;
  vector<array<int, 3>> tri;
  delaunay() : pool(nullptr), n(0) {}
  delaunay(vector<T> &px, vector<T> &py) : pool(nullptr), n(0) { build(px, py); }
  void build(vector<T> &px, vector<T> &py) {
    n = (int)px.size(); tri.clear(); pool = nullptr;
    if (n < 2) return;
    vector<int> ord(n); iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int a, int b) {
      return px[a]<px[b] || (px[a]==px[b] && py[a]<py[b]);
    });
```

Delaunay (cont.)

```
    vector<pt> sp(n); vector<int> si(n);
    for (int i = 0; i < n; i++) { sp[i] = {px[ord[i]], py[ord[i]]}; si[i] = ord[i]; }
    Quad* e = rec(sp, si, 0, n).first;
    vector<Quad*> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
    auto add = [&](Quad* e) {
      Quad* c = e; vector<int> face;
      do { c->mark=1; face.push_back(c->id); q.push_back(c->r()); c=c->next(); } while (c!=e);
      if ((int)face.size()==3) tri.push_back({face[0],face[1],face[2]});
    };
    add(e);
    while (qi < (int)q.size()) if (!(e=q[qi++])->mark) add(e);
  }
};
```

Halfplane-intersection

```
/*
 *Description:* Halfplane intersection
 *Status:* Partially tested
*/
const ld inf = 1e18;
const ld eps = 1e-9;
vector<point2d<ld>> hp_intersect(vector<halfplane> &H) {
  point2d<ld> box[4] = {{inf, inf}, {-inf, inf}, {-inf, -inf}, {inf, -inf}};
  for (int i = 0; i < 4; i++) {
    halfplane aux(box[i], box[(i + 1) % 4]);
    H.push_back(aux);
  }
  sort(H.begin(), H.end());
  deque<halfplane> dq;
  int len = 0;
  for (int i = 0; i < H.size(); i++) {
    while (len > 1 && H[i].out(inter(dq[len - 1], dq[len - 2])))
      dq.pop_back(), --len;
    while (len > 1 && H[i].out(inter(dq[0], dq[1])))
      dq.pop_front(), --len;
    if (len > 0 && fabsl((H[i].pq ^ dq[len - 1].pq)) < eps) {
      if ((H[i].pq | dq[len - 1].pq) < 0.0)
        return vector<point2d<ld>>();
      if (H[i].out(dq[len - 1].p))
        dq.pop_back(), --len;
      else
        continue;
    }
    dq.push_back(H[i]), ++len;
  }
  while (len > 2 && dq[0].out(inter(dq[len - 1], dq[len - 2])))
    dq.pop_back(), --len;
  while (len > 2 && dq[len - 1].out(inter(dq[0], dq[1])))
    dq.pop_front(), --len;
  if (len < 3)
    return vector<point2d<ld>>();
  vector<point2d<ld>> ret(len);
  for (int i = 0; i + 1 < len; i++)
    ret[i] = inter(dq[i], dq[i + 1]);
  ret.back() = inter(dq[len - 1], dq[0]);
  return ret;
}
```

Halfplane

```
/*
 *Description:* Halfplane
 *Status:* Partially tested
*/
struct halfplane {
  point2d<ld> p, pq;
  ld angle;
  halfplane() {}
  halfplane(point2d<ld> &a, point2d<ld> &b) : p(a), pq(b - a) {
    angle = atan2l(pq.y, pq.x);
  }
  bool out(point2d<ld> r) { return ((pq) ^ (r - p)) < -eps; }
  bool operator<(halfplane &e) { return angle < e.angle; }
  friend point2d<ld> inter(halfplane &s, halfplane &t) {
    ld alpha = ((t.p - s.p) ^ t.pq) / ((s.pq) ^ (t.pq));
    return s.p + (s.pq * alpha);
  }
};
```

Hull-diameter

```cpp
/*
 *Description:* Returns the two points with max distance on a convex hull (CCW, no collinear points). $O(n)$. Uses point2d.
 *Status:* KACTL based, not self tested
*/
template<class T>
array<point2d<T>,2> hull_diameter(vector<point2d<T>>& S) {
  int n = S.size(), j = n < 2 ? 0 : 1;
  pair<T, array<point2d<T>,2>> res = {0, {S[0], S[0]}};
  for (int i = 0; i < j;)
    for (;; j = (j+1)%n) {
      res = max(res, {S[i].sqdist(S[j]), {S[i], S[j]}});
      if (((S[(j+1)%n]-S[j])^(S[i+1]-S[i])) >= 0)
        { i++; break; }
    }
  return res.second;
}
```

In-convex

```cpp
/*
 *Description:* Point inside convex polygon (CCW, lowest-leftmost first)
 *Returns:* 1 inside, 0 border, -1 outside
 *Complexity:* $O(log n)$
 */
template <typename T> int in_convex(vector<point2d<T>> &P, point2d<T> q) {
  int n = P.size();
  if (n < 3) return -1;
  point2d<T> q0 = q - P[0];
  T c1 = (P[1] - P[0]) ^ q0, c2 = (P[n - 1] - P[0]) ^ q0;
  if (c1 < 0 || c2 > 0) return -1;
  int l = 1, r = n - 1;
  while (r - l > 1) {
    int mid = (l + r) / 2;
    ((P[mid] - P[0]) ^ q0) >= 0 ? l = mid : r = mid;
  }
  T c = (P[l + 1] - P[l]) ^ (q - P[l]);
  return c < 0 ? -1 : (c == 0 || c1 == 0 || c2 == 0 ? 0 : 1);
}
```

Lattice-point

```cpp
/*
 *Description:* How many integers point are inside or in the bounds of the polygon
 - Only works for integers points
 *Status:* Tested on CSES
*/
pair<ll, ll> lattice_points(vector<point2d<ll>> &P) {
  P.push_back(P.front());
  ll area = 0, bounds = 0;
  for(int i = 0; i < (int)P.size()-1; ++i) {
    area += P[i]^(P[i+1]);
    point2d<ll> p = P[i+1]-P[i];
    bounds += abs(__gcd(p.x, p.y));
  }
  ll inside = (abs(area) - bounds + 2)/2;
  // Dejar el poligono como estaba antes
  P.pop_back();
  return {inside, bounds};
}
```

Line-distance

```cpp
/*
 *Description:* Signed distance from point $p$ to line through $a$, $b$. Positive on left side seen from $a$ to $b$. Uses point2d.
 *Status:* KACTL based, not self tested
*/
template<class T>
ld line_dist(point2d<T> a, point2d<T> b, point2d<T> p) {
  return (ld)((b-a)^(p-a)) / (b-a).abs();
}
```

Minimum-enclosing-circle

```cpp
/*
 *Description:* Minimum enclosing circle, Welzl algorithm $O(n)$ expected
 *Status:* Tested
 */
circle<ld> minimum_enclosing_circle(vector<point2d<ld>> p) {
  mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
  shuffle(p.begin(), p.end(), rng);
  int n = p.size();
  if (n == 0) return circle<ld>();
  circle<ld> cur(p[0], 0);
```

Minimum-enclosing-circle (cont.)

```cpp
  auto out = [&](int i) { return sign(sqrtl(cur.c.sqdist(p[i])) - cur.r) > 0; };
  for (int i = 1; i < n; i++) if (out(i)) {
    cur = circle<ld>(p[i], 0);
    for (int j = 0; j < i; j++  ) if (out(j)) {
      cur = circle<ld>((p[i] + p[j]) / 2, sqrtl(p[i].sqdist(p[j])) / 2);
      for (int k = 0; k < j; k++) if (out(k))
        cur = circle<ld>(p[i], p[j], p[k]);
    }
  }
  return cur;
}
```

Minimum-enclosing-sphere

```cpp
/*
 *Description:* Smallest Enclosing Sphere using simulated annealing
 *Time:* $O("iterations" dot n)$, roughly $O(n)$ with high constant
 *Status:* Tested
 */

template <typename T>
sphere<T> minimum_enclosing_sphere(vector<point3d<T>> p, int it = 100000) {
  int n = p.size();
  if (n == 0) return sphere<T>();
  if (n == 1) return sphere<T>(p[0], 0);

  point3d<T> c(0, 0, 0);
  for (int i = 0; i < n; i++)
    c = c + p[i];
  c = c / (ld)n;

  ld ratio = 0.1;
  while (it--) {
    int pos = 0;
    for (int i = 1; i < n; i++) {
      if (c.sq_dist(p[i]) > c.sq_dist(p[pos]))
        pos = i;
    }
    c = c + (p[pos] - c) * ratio;
    ratio *= 0.998;
  }
  return sphere(c, c.dist(p[pos]));
}
```

Minkowski

```cpp
/*
 *Description:* Minkowski sum of two CONVEX polygons (handles degenerate)
 *Complexity:* $O(n + m)$
*/
template <typename T>
vector<point2d<T>> minkowski(vector<point2d<T>> &p, vector<point2d<T>> &q) {
  vector<point2d<T>> v;
  if (min(p.size(), q.size()) < 3) {
    for (auto &a : p) for (auto &b : q)
      v.push_back(a + b);
    return convex_hull(v);
  }
  auto reorder = [](vector<point2d<T>> &v) {
    if (((v[1] - v[0]) ^ (v[2] - v[0])) < 0)
      reverse(v.begin(), v.end());
    int pos = 0;
    for (int i = 1; i < (int)v.size(); i++)
      if (pair{v[i].y, v[i].x} < pair{v[pos].y, v[pos].x})
        pos = i;
    rotate(v.begin(), v.begin() + pos, v.end());
  };
  reorder(p), reorder(q);
  for (int i = 0; i < 2; i++)
    p.push_back(p[i]), q.push_back(q[i]);
  int i = 0, j = 0;
  while (i + 2 < p.size() or j + 2 < q.size()) {
    v.push_back(p[i] + q[j]);
    T c = (p[i + 1] - p[i]) ^ (q[j + 1] - q[j]);
    if (c >= 0 and i + 2 < p.size()) i++;
    if (c <= 0 and j + 2 < q.size()) j++;
  }
  return v;
}
```

Nearest-2-points

```cpp
/*
 *Description:* Using divide and conquer, allow us to know what are the two nearest point between them
 *Status:* Tested on CSES
*/
```

Nearest-2-points (cont.)

```cpp
#define sq(x) ((x)*(x))
template <typename T>
pair<point2d<T>, point2d<T>> nearest_points(vector<point2d<T>> &P, int l, int r) {
  const T INF = 1e10;
  if (r-l == 1) return {P[l], P[r]};
  if (l >= r) return {{INF, INF}, {-INF, -INF}};

  int m = (l+r)/2;
  pair<point2d<T>, point2d<T>> D1, D2, D;
  D1 = nearest_points(P, l, m);
  D2 = nearest_points(P, m+1, r);
  D = (D1.first.sqdist(D1.second) <= D2.first.sqdist(D2.second) ? D1 : D2);

  T d = D.first.sqdist(D.second), x_center = (P[m].x + P[m+1].x)/2;
  vector<point2d<T>> Pk;
  for (int i = l; i <= r; i++)
    if (sq(P[i].x-x_center) <= d)
      Pk.push_back(P[i]);

  sort(Pk.begin(), Pk.end(), [](const point2d<T> p1, const point2d<T> p2) {
    return p1.y != p2.y ? p1.y < p2.y : p1.x < p2.x;
  });

  for(int i = 0; i < Pk.size(); ++i) {
    for(int j = i-1; j >= 0; --j) {
      if(sq(Pk[i].y-Pk[j].y) > d) break;
      if(Pk[i].sqdist(Pk[j]) <= D.first.sqdist(D.second))
        D = {Pk[i],Pk[j]};
    }
    for(int j = i+1; j < Pk.size(); ++j) {
      if(sq(Pk[i].y-Pk[j].y) > d) break;
      if(Pk[i].sqdist(Pk[j]) <= D.first.sqdist(D.second))
        D = {Pk[i],Pk[j]};
    }
  }

  return D;
}

template <typename T>
pair<point2d<T>, point2d<T>> nearest_points(vector<point2d<T>> &P) {
  sort(P.begin(), P.end(), [](const point2d<T> &p1, const point2d<T> &p2) {
    if (p1.x == p2.x) return p1.y < p2.y;
    return p1.x < p2.x;
  });

  return nearest_points(P, 0, P.size()-1);
}
```

Order-by-angle

```cpp
/*
 *Description:* Sort the point depending their angle
 *Status:* Not tested
*/
template <typename T>
int semiplane(point2d<T> p) { return p.y > 0 or (p.y == 0 and p.x > 0); }

template <typename T>
void order_by_angle(vector<point2d<T>> &P) {
  sort(P.begin(), P.end(), [](point2d<T> &p1, point2d<T> &p2) {
    int s1 = semiplane(p1), s2 = semiplane(p2);
    return s1 != s2 ? s1 > s2 : (p1^p2) > 0;
  });
}
```

Order-by-slope

```cpp
/*
 *Description:* Sort the points depending the angle
 *Status:* Not tested
*/
template <typename T>
void order_by_slope(vector<point2d<T>> &P) {
  sort(P.begin(), P.end(), [](const point2d<T> &p1, const point2d<T> &p2) {
    Fraction<T> r1 = Fraction<T>(p1.x, p1.y), r2 = Fraction<T>(p2.x, p2.y);
    return r2 < r1;
  });
}
```

Point-2d

```cpp
/*
 *Description:* Just a 2D Point
 *Status:* Tested
 */
template <typename T> struct point2d {
  T x, y;
  point2d(){};
  point2d(T x_, T y_) : x(x_), y(y_) {}
  point2d<T> &operator+=(point2d<T> t) {
    x += t.x, y += t.y;
    return *this;
  }
  point2d<T> operator-(point2d<T> t) { return {x - t.x, y - t.y}; }
  point2d<T> operator+(point2d<T> t) { return {x + t.x, y + t.y}; }
  point2d<T> operator*(T t) { return {x * t, y * t}; }
  point2d<T> operator/(T t) { return {x / t, y / t}; }
  point2d<T> &operator-=(point2d<T> t) {
    x -= t.x, y -= t.y;
    return *this;
  }
  T operator|(point2d<T> b) { return x * b.x + y * b.y; }
  T operator^(point2d<T> b) { return x * b.y - y * b.x; }
  T cross(point2d<T> a, point2d<T> b) { return (a - *this) ^ (b - *this); }
  T norm() { return (*this) | (*this); }
  T sqdist(point2d<T> b) { return ((*this) - b).norm(); }
  double abs() { return sqrt(norm()); }
};
template <typename T> point2d<T> operator*(T a, point2d<T> b) { return b * a; }
```

Point-3d

```cpp
/*
 *Description:* 3D Point with full support
 *Status:* Tested
 */
template <typename T> struct point3d {
  T x, y, z;
  point3d() : x(0), y(0), z(0) {}
  point3d(T x_, T y_, T z_) : x(x_), y(y_), z(z_) {}

  point3d<T> &operator=(const point3d<T> &t) {
    x = t.x, y = t.y, z = t.z;
    return *this;
  }
  point3d<T> &operator+=(const point3d<T> &t) {
    x += t.x, y += t.y, z += t.z;
    return *this;
  }
  point3d<T> &operator-=(const point3d<T> &t) {
    x -= t.x, y -= t.y, z -= t.z;
    return *this;
  }
  point3d<T> &operator*=(T t) {
    x *= t, y *= t, z *= t;
    return *this;
  }
  point3d<T> &operator/=(T t) {
    x /= t, y /= t, z /= t;
    return *this;
  }
  point3d<T> operator-(const point3d<T> &t) const {
    return {x - t.x, y - t.y, z - t.z};
  }
  point3d<T> operator+(const point3d<T> &t) const {
    return {x + t.x, y + t.y, z + t.z};
  }
  point3d<T> operator*(T t) const { return {x * t, y * t, z * t}; }
  point3d<T> operator/(T t) const { return {x / t, y / t, z / t}; }

  // Dot product
  T dot(const point3d<T> &b) const { return x * b.x + y * b.y + z * b.z; }
  T operator|(const point3d<T> &b) const { return dot(b); }

  // Cross product (returns a vector in 3D)
  point3d<T> cross(const point3d<T> &b) const {
    return {y * b.z - z * b.y, z * b.x - x * b.z, x * b.y - y * b.x};
  }
  point3d<T> operator^(const point3d<T> &b) const { return cross(b); }

  // Signed volume of tetrahedron (a, b, c, *this as origin)
  T triple(const point3d<T> &a, const point3d<T> &b) const {
    return dot(a ^ b);
  }

  bool nonzero() const { return x != 0 || y != 0 || z != 0; }
};
```

Point-3d (cont.)

```
  T norm() const { return dot(*this); }
  T sq_dist(const point3d<T> &b) const { return (*this - b).norm(); }
  double abs() const { return sqrt((double)norm()); }
  double dist(const point3d<T> &b) const { return (*this - b).abs(); }

  point3d<T> unit() const { return *this / this->abs(); }

  double proj(const point3d<T> &b) const { return (double)dot(b) / b.abs(); }
  double angle(const point3d<T> &b) const {
    return acos(clamp((double)dot(b) / this->abs() / b.abs(), -1.0, 1.0));
  }

  // Rotations around axes
  point3d<T> rotate_z(double a) const {
    return {(T)(cos(a) * x - sin(a) * y), (T)(sin(a) * x + cos(a) * y), z};
  }
  point3d<T> rotate_x(double a) const {
    return {x, (T)(cos(a) * y - sin(a) * z), (T)(sin(a) * y + cos(a) * z)};
  }
  point3d<T> rotate_y(double a) const {
    return {(T)(cos(a) * x + sin(a) * z), y, (T)(-sin(a) * x + cos(a) * z)};
  }
};

template <typename T> point3d<T> operator*(T a, point3d<T> b) { return b * a; }
```

Point-inside-polygon

```
/*
 *Description:* Check if a point is inside, bounds or out of the polygon
 - 0: Bounds
 - 1: Inside
 - -1: Out
 *Status:* Tested
*/
template<typename T>
int point_inside_polygon(vector<point2d<T>> &P, point2d<T> q) {
  int N = P.size(), cnt = 0;
  for(int i = 0; i < N; i++) {
    int j = (i == N-1 ? 0 : i+1);
    segment<T> s(P[i], P[j]);
    if(s.contain(q)) return 0;
    if(P[i].x <= q.x and q.x < P[j].x and q.cross(P[i], P[j]) < 0) cnt++;
    else if(P[j].x <= q.x and q.x < P[i].x and q.cross(P[j], P[i]) < 0) cnt++;
  }
  return cnt&1 ? 1 : -1;
}
```

Polygon-area

```
/*
 *Description:* Get the area of a polygon
 - If you want the double of the area, just enable the flag
 *Status:* Highly tested
*/
template<typename T>
T polygon_area(vector<point2d<T>> P, bool x2 = 0) {
  if (P.size() < 3) return (T)0;
  T area = 0;
  for(int i = 0; i < P.size()-1; ++i)
    area += P[i]^(P[i+1]);
  // Si el primer punto se repite, sacar:
  area += (P.back())^(P.front());
  return abs(area)/ (x2 ? 1 : 2);
}
```

Polygon-center

```
/*
 *Description:* Returns the centroid (center of mass) of a polygon. $O(n)$. Uses point2d.
 *Status:* KACTL based, not self tested
*/
using P = point2d<ld>;
P polygon_center(vector<P>& v) {
  P res(0, 0); ld A = 0;
  int n = v.size();
  for (int i = 0, j = n-1; i < n; j = i++) {
    res = res + (v[i]+v[j]) * (v[j]^v[i]);
    A += v[j]^v[i];
  }
  return res / A / 3;
```

Polygon-center (cont.)

```
}
```

Polygon-cut

```
/*
 *Description:* Returns polygon with everything to the left of line $s -> e$ cut away. Uses point2d.
 *Status:* KACTL based, not self tested
*/
using P = point2d<ld>;
vector<P> polygon_cut(vector<P>& poly, P s, P e) {
  vector<P> res;
  int n = poly.size();
  for (int i = 0; i < n; i++) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev-cur) * (a / (a-b)));
    if (a < 0) res.push_back(cur);
  }
  return res;
}
```

Segment-distance

```
/*
 *Description:* Shortest distance from point $p$ to segment $s$-$e$. Uses point2d.
 *Status:* KACTL based, not self tested
*/
using P = point2d<ld>;
ld seg_dist(P s, P e, P p) {
  if (s.sqdist(e) < 1e-18) return (p-s).abs();
  auto d = (e-s).norm();
  auto t = min(d, max((ld)0.0, (p-s)|(e-s)));
  return ((p-s)*d - (e-s)*t).abs() / d;
}
```

Segment

```
/*
 *Description:* Segment implementation
 *Status:* Partially tested
*/
template<typename T>
struct segment {
  point2d<T> P;
  point2d<T> Q;
  const T INF = numeric_limits<T>::max();
  segment(point2d<T> P, point2d<T> Q): P(P), Q(Q) {}
  int sign(T x, T eps = 0) { return x > eps ? 1 : x < -eps ? -1 : 0; }
  bool contain(point2d<T> p, T eps = 0) {
    return ((P - p)|(Q - p)) <= (T)0 and abs(((Q - P)^(p - P))) <= eps;
  }
  bool intersect(segment<T> b) {
    if (this->contain(b.P) or this->contain(b.Q) or b.contain(P) or b.contain(Q))
        return true;
    // change < 0 or <= depending the problem
    return sign((((b.P) - P))^((Q - P)))*sign(((b.Q - P)^(Q - P))) < 0 and
           sign(((P - b.Q)^(b.Q - b.P)))*sign(((Q- b.P)^(b.Q - b.P))) < 0;
  }
  // not tested
  point2d<T> intersection(segment<T> b) {
    if(this->intersect(b))
      return (((b.Q-b.P)^(Q-b.P))*P + ((P-b.P)^(b.Q-b.P))*Q)/((P-Q)^(b.Q-b.P));
    return {INF, INF};
  }
};
```

Sphere

```
/*
 *Description:* 3D Sphere defined by center point and radius.
 Supports containment testing with epsilon tolerance.
 *Status:* Tested
*/
template <typename T> struct sphere {
  point3d<T> center;
  ld radius;
  sphere() : center(), radius(0) {}
  sphere(point3d<T> c, ld r) : center(c), radius(r) {}
  bool contains(const point3d<T> &pt) const {
    return center.sq_dist(pt) <= radius * radius + 1e-9;
  }
};
```

Formulario

1. Notación

- N, Z, Q, R, C: naturales (incl. 0 cuando se indique), enteros, racionales, reales, complejos. F_p: cuerpo finito.
- a | b: a divide a b. a == b (mod m): m | (a-b).
- v_p(n) = max{k : p^k | n} (valuación p-ádica).
- phi(n) totient, mu(n) Möbius, d(n) divisores, sigma(n) suma de divisores.
- (a/p) Legendre, (a/n) Jacobi, ord_n(a) orden multiplicativo.
- (Z/nZ)^*: unidades módulo n; |*| cardinalidad.
- [P]: indicador (1 si P verdadero, 0 si no).
- x^(n_dn) = x(x-1)...(x-n+1) (factorial descendente); $x^(n_up) = x(x+1)...(x+n-1)$ (ascendente).
- H_n = sum_(k=1..n) 1/k número armónico, H_0 = 0.
- [x^n]F(x): coeficiente de x^n en F.

2.1 Algoritmos básicos

- Euclides: gcd(a,b) = gcd(b, a mod b), gcd(a,0)=a. Costo O(log min(a,b)).
- Euclides extendido: devuelve (g,x,y) con g=gcd(a,b) y ax+by=g. Si gcd(a,m)=1, x mod m = a^(-1).
- Exponenciación rápida: a^n mod m en O(log n) por cuadrados sucesivos.
- Criba de Eratóstenes: O(N log log N). Criba linear: O(N). Segmentada: O((R-L)log log R + sqrtR).

2.2 Inverso modular y congruencias lineales

- a^(-1) (mod m) existe <=> gcd(a,m)=1.
- ax == b (mod n) tiene solución <=> d=gcd(a,n) | b. Si sí, hay d soluciones módulo n: x = x_0 + k(n/d), k=0,...,d-1.
- Inverso sin extgcd: a^(-1) == a^(phi(m)-1) (mod m) (si m primo: a^(m-2)).

2.3 Invariantes del Euclides extendido

Al correr extgcd(a,b) mantenemos (r_i,s_i,t_i) con r_i = s_i a + t_i b y las recurrencias

r_i = r_(i-2) - q_i r_(i-1), s_i = s_(i-2) - q_i s_(i-1), t_i = t_(i-2) - q_i t_(i-1).

Inicialización: (a,1,0),(b,0,1). Al final r_k = gcd(a,b), y los coeficientes de Bézout son s_k,t_k. Invariante: s_i t_(i-1) - s_(i-1) t_i = (-1)^(i+1).

2.4 Teorema Chino del Resto (CRT)

Teorema (CRT). Si m_1,...,m_k son coprimos dos a dos y M=prod m_i. el sistema x == a_i (mod m_i) tiene solución única módulo M: x = sum_(i=1..k) a_i M_i v_i mod M. M_i = M/m_i. v_i == M_i^(-1) (mod m_i).

CRT no coprimo: consistente <=> a_i == a_j (mod gcd(m_i,m_j)) para todo i,j. Se resuelve combinando pares (módulo resultante = lcm).

2.5 Teoremas clásicos

- Fermat: p primo, p !| a => a^(p-1) == 1 (mod p).
- Euler: gcd(a,n)=1 => a^(phi(n)) == 1 (mod n).
- Wilson: (p-1)! == -1 (mod p) <=> p primo. Generalizado: $(n-1)! == -1 (mod n) <=> n=1,4 o n$ primo.

2.6 Funciones aritméticas

- phi(n) = n prod_(p | n)(1-1/p); sum_(d | n) phi(d) = n.
- mu(n)=0 si p^2 | n; si no, mu(n)=(-1)^(omega(n)) (omega(n)= #primos distintos). sum_(d | n) mu(d) = [n=1].
- Inversión de Möbius: $g(n) = sum_(d | n) f(d) <=> f(n) = sum_(d | n) mu(d) g(n/d), i.e. f = g * mu$.
- n = prod p_i^(e_i) => d(n) = prod(e_i+1), $sigma(n) = prod (p_i^(e_i+1)-1)/(p_i-1)$.

2.7 Residuos cuadráticos, Legendre, Jacobi

- Euler: a es QR mód p primo impar <=> a^((p-1)/2) == 1; QNR $<=> == -1$.
- #QR mód p: (p-1)/2. Multiplicatividad: (ab/p) = (a/p)(b/p).
- Reciprocidad cuadrática: (p/q)(q/p) = (-1)^(((p-1)/2)((q-1)/2)).
- Complementarios: (-1/p) = (-1)^((p-1)/2), (2/p) = (-1)^((p^2-1)/8).
- Jacobi: (a/n) = prod (a/p_i)^(e_i), n = prod p_i^(e_i) impar. $(a/n) = 1 **no** implica QR mód n si n$ compuesto.
- Reciprocidad Jacobi: (m/n)(n/m) = (-1)^(((m-1)/2)((n-1)/2)) para m,n impares coprimos.

2.8 Tonelli-Shanks y raíces modulares

Para a QR mód p primo impar:

- p == 3 (mod 4): x == a^((p+1)/4).
- p == 5 (mod 8): $x == a^((p+3)/8) * (a^((p-1)/4) == 1 ? 1 : 2^((p-1)/4))$.
- p == 1 (mod 8): algoritmo general Tonelli-Shanks, O(log^2 p).

Discrete root x^k == a (mod p): si gcd(k,p-1)=1, $x = a^(k^(-1) mod (p-1))$; si no, reducir a log discreto.

2.9 Raíz primitiva y estructura de $(Z/nZ)^*$

- g primitiva mód n <=> ord_n(g) = phi(n).
- Existe <=> n in {1,2,4,p^k,2p^k} (p primo impar). En ese caso hay phi(phi(n)) primitivas.
- Estructura general: si n = 2^e prod p_i^(e_i), (Z/nZ)^* ~= (Z/2Z)^a x (Z/2^(e-2)Z) x prod Z/(p_i^(e_i-1)(p_i-1))Z con a = 0,1,2 para e<2, e=2, e >= 3.

2.10 Logaritmo discreto (BSGS)

Resolver g^x == h (mod n), gcd(g,n)=1. Con m=ceil(sqrtn), escribir $x = im+j, 0 <= j < m$; precalcular g^j, buscar h * (g^(-m))^i$. Costo O(sqrtn).

2.11 Primalidad y factorización

- Miller-Rabin: n-1 = d 2^s (d impar); test: a^d == 1 o a^(d 2^r) == -1 para algún r<s. Bases seguras 64 bits: {2,3,5,7,11,13,17,19,23,29,31,37}.
- Pollard Rho: f(x)=x^2+c, Floyd para ciclo; factor gcd(|x-y|,n). O(n^(1/4)) esperado.

2.12 Teoremas avanzados

Lifting the Exponent (LTE). p primo impar. p | x-y. p !| xy: $v_p(x^n - y^n) = v_p(x-y) + v_p(n)$. Para p=2. x,y impares: v_2(x^n - y^n) = v_2(x-y) si n impar: v_2(x+y)+v_2(x-y)+v_2(n)-1 si n par. También p | x+y, n impar: v_p(x^n+y^n) = v_p(x+y)+v_p(n)$.

Lucas. m = sum m_i p^i. n = sum n_i p^i: $C(m,n) == prod C(m_i, n_i) (mod p)$.

Kummer. v_p(C(m,n)) = #acarreos al sumar n y m-n en base p.

3.1 Coeficientes y binomios

- C(n,k) = n! / (k!(n-k)!). C(n; n_1,...,n_r) = n! / (n_1!...n_r!).
- Newton generalizado: (1+x)^alpha = sum_(n >= 0) C(alpha,n) x^n, con C(alpha,n) = alpha(alpha-1)...(alpha-n+1)/n! (válido para alpha in R formalmente).
- Vandermonde: C(m+n, k) = sum_i C(m,i) C(n, k-i).
- Hockey-stick: sum_(i=r..n) C(i,r) = C(n+1, r+1).
- Recurrencia: C(n,k) = C(n-1, k-1) + C(n-1, k).
- Identidades: sum_k C(n,k) = 2^n; sum_k C(n,k)^2 = C(2n, n); $sum_k k C(n,k) = n 2^(n-1); sum_k (-1)^k C(n,k) = [n=0]$.
- C(n,k) == C(n mod p, k mod p) C(floor(n/p), floor(k/p)) (mod p) (Lucas).

3.2 Stars and bars

Soluciones de x_1+...+x_k=n:

- x_i >= 0: C(n+k-1, k-1).
- x_i >= 1: C(n-1, k-1).
- x_i >= a_i: sustituir x_i' = x_i - a_i.

3.3 Números especiales

- Catalan: C_n = C(2n,n)/(n+1), C_(n+1) = sum C_i C_(n-i), C_0 = 1.
- Stirling 2a: S(n,k) = (1/k!) sum_(j=0..k) (-1)^(k-j) C(k,j) j^n; $S(n,k) = S(n-1,k-1) + k S(n-1,k)$. Inversión: x^n = sum S(n,k) x^(k_dn)$.
- Stirling 1a (sin signo) c(n,k): x^(n_up) = sum c(n,k)x^k; $c(n,k) = c(n-1,k-1)+(n-1)c(n-1,k)$. Con signo s(n,k) = (-1)^(n-k)c(n,k), x^(n_dn) = sum s(n,k)x^k$.
- Bell: B_n = sum_k S(n,k); B_(n+1) = sum_k C(n,k) B_k; EGF exp(e^x-1).
- Eulerianos A(n,k): permutaciones con k ascensos; $A(n,k) = (n-k)A(n-1,k-1)+(k+1)A(n-1,k)$, sum_k A(n,k) = n!. Explícito: A(n,k) = sum_(j=0..k+1) (-1)^j C(n+1,j)(k+1-j)^n$.
- Derangements: D_n = n! sum_(k=0..n) (-1)^k / k!, $D_n = (n-1)(D_(n-1)+D_(n-2)), D_n = n D_(n-1)+(-1)^n$.
- Particiones p(n): OGF prod_(k >= 1) 1/(1-x^k) = sum p(n)x^n. Euler pentagonal: prod_(k >= 1)(1-x^k) = sum_(j in Z) (-1)^j x^(j(3j-1)/2). Asintótico p(n) ~ e^(pisqrt(2n/3)) / (4nsqrt3).
- Números de Lah L(n,k) = C(n-1, k-1) n! / k! (particiones ordenadas).
- Números de Narayana N(n,k) = C(n,k) C(n,k-1) / n (permutaciones con k runs).
- Números de Schröder (caminos con pasos (1,0),(0,1),(1,1)): OGF $S(x) = (1-x-sqrt(x^2-6x+1))/(2x)$.
- Números de Motzkin M_n: pasos {(1,0),(1,1),(1,-1)} sin bajar; $sum M_n x^n = (1-x-sqrt(1-2x-3x^2))/(2x^2)$.
- Fubini (ordenados Bell) F_n = sum_k k! S(n,k); EGF 1/(2-e^x).
- Fibonacci: F_n = (phi^n-psi^n)/sqrt5, phi = (1+sqrt5)/2, $psi = (1-sqrt5)/2. sum_(i=0..n) F_i = F_(n+2)-1; sum F_i^2 = F_n F_(n+1)$.
- Lucas: L_n = F_(n-1)+F_(n+1) = phi^n+psi^n.

3.4 Inclusión-exclusión

|U_(i=1..n) A_i| = sum|A_i| - sum|A_i cap A_j| + ... + (-1)^(n-1)|A_1 cap ... cap A_n|.

Forma general: si A_i es el evento de "tener propiedad i", entonces

#{sin ninguna propiedad} = sum_(S subset [n]) (-1)^(|S|) | cap_(i in S) A_i|.

3.5 Sumas de potencias

- sum_(k=1..n) k = n(n+1)/2.
- sum_(k=1..n) k^2 = n(n+1)(2n+1)/6.
- sum_(k=1..n) k^3 = (n(n+1)/2)^2.
- Faulhaber: $sum_(k=1..n) k^p = (1/(p+1)) sum_(j=0..p) (-1)^j C(p+1,j) B_j n^(p+1-j) con B_j$ números de Bernoulli.
- B_0=1, B_1=-1/2, B_2=1/6, B_4=-1/30, B_6=1/42. OGF: $x/(e^x-1) = sum B_n x^n/n!$.

4.1 Recurrencias lineales homogéneas

a_n = c_1 a_(n-1) + ... + c_d a_(n-d). Polinomio característico P(lambda) = lambda^d - c_1 lambda^(d-1) - ... - c_d.

- Si las raíces lambda_i son simples: a_n = sum alpha_i lambda_i^n, coeficientes de condiciones iniciales.
- Raíz lambda de multiplicidad m: contribución $(beta_0+beta_1 n+...+beta_(m-1)n^(m-1))lambda^n$.
- Inhomogénea a_n = sum c_i a_(n-i) + f(n): suma la solución particular (polinomio si f polinomial; lambda^n * poly(n) si f exponencial).

4.2 Exponenciación de matrices

Para a_n con recurrencia de orden d, escribir $v_n = (a_n, a_(n-1), ..., a_(n-d+1))^T = A v_(n-1)$, con

A = [c_1, c_2, ..., c_d; 1, 0, ..., 0; :, ..., :; 0, ..., 1, 0].

Entonces v_n = A^(n-d+1) v_(d-1), calculado por exponenciación binaria en O(d^3 log n) (o O(d^2 log n) con Berlekamp-Massey + Kitamasa).

4.3 Berlekamp-Massey

Dada una secuencia s_0, s_1, ... generada por una recurrencia lineal de orden <= L, encuentra la recurrencia mínima en O(n^2). Permite calcular s_N con Kitamasa en O(L^2 log N) o O(L log L log N) con FFT. Condición: el orden es menor a ~ n/2.

4.4 Transformaciones lineales

- T: R^n -> R^m lineal $<=> T(alpha x + beta y) = alpha T(x) + beta T(y)$.
- Se representa por matriz A in R^(mxn) con T(x) = Ax.
- Composición: T_2 o T_1 <-> A_2 A_1. Inversa: T^(-1) <-> A^(-1) (si det A != 0).
- Cambio de base P: A' = P^(-1) A P. det A' = det A, tr A' = tr A, espectro invariante.

4.5 Determinantes

- det A = sum_(sigma in S_n) sgn(sigma) prod_i A_(i,sigma(i)).
- Propiedades: multilineal por filas, det A = det A^T, $det(AB) = det A det B, det(A^(-1)) = 1/det A$.
- Cramer: x_i = det A_i / det A, A_i = A con columna i reemplazada por b.
- Cofactor: C_(ij) = (-1)^(i+j) M_(ij) (M_(ij) = determinante al tachar fila i, col j); det A = sum_j A_(ij) C_(ij). A^(-1)_(ij) = C_(ji)/det A.
- Vandermonde: det(x_i^(j-1))_(i,j) = prod_(i<j)(x_j-x_i).
- Cauchy: $det(1/(x_i+y_j)) = prod_(i<j)(x_j-x_i)(y_j-y_i) / prod_(i,j)(x_i+y_j)$.
- Circulante: si c = (c_0,...,c_(n-1)), $det C = prod_(k=0..n-1)(sum_j c_j omega^(jk)) con omega = e^(2pii/n)$.
- Lema de matriz: det(I+uv^T) = 1+v^Tu.
- Sylvester: det(I_m+AB) = det(I_n+BA) (compatibilidad adecuada).

5.1 Expansión

x = a_0 + 1/(a_1 + 1/(a_2 + ...)), a_i in Z, a_i > 0 para i >= 1.

Se denota [a_0; a_1, a_2, ...]. Algoritmo: a_0 = floor(x), $x <- 1/(x-a_0)$.

5.2 Convergentes

p_(-2)=0, p_(-1)=1, q_(-2)=1, q_(-1)=0; recurrencias

p_i = a_i p_(i-1) + p_(i-2), q_i = a_i q_(i-1) + q_(i-2).

El convergente p_i/q_i satisface p_i q_(i-1) - p_(i-1) q_i = (-1)^(i-1). Es una fracción reducida y la mejor aproximación racional con denominador <= q_i.

5.3 Aplicaciones

- Ecuación de Pell x^2 - Ny^2 = 1: soluciones = convergentes de sqrtN con periodo par.
- Aproximación racional de x: truncar en q_i <= B.
- Stern-Brocot: árbol que contiene todo Q_(>0); descenso por comparación con mediana.

6.1 Operaciones básicas

Sobre R[x], R cuerpo (o dominio):

6.1 Operaciones básicas (cont.)

- División: A = BQ + R, deg R < deg B. GCD polinomial por Euclides.
- Euclides extendido polinomial: A * s + B * t = gcd(A,B).

6.2 Teorema de Bézout

Bézout para polinomios. gcd(A,B)=1 en K[x] <=> exists s,t in K[x] con As+Bt=1.

Raíz alpha de A <=> (x-alpha) | A. Múltiples raíces <=> gcd(A,A') no constante.

6.3 Resultante y discriminante

La matriz de Sylvester S(A,B) de A = sum_(i=0..m) a_i x^i, $B = sum_(j=0..n) b_j x^j es la (m+n) x (m+n) con filas de coeficientes de A$ y B desplazados. Se define

Res(A,B) = det S(A,B).

Propiedades: $Res(A,B) = (-1)^(mn) Res(B,A) = a_m^n prod_(B(beta_i)=0) A(beta_i). Además A,B tienen raíz común <=> Res(A,B) = 0$.

Discriminante: disc(A) = (-1)^(C(m,2)) a_m^(-1) Res(A,A').

6.4 Interpolación (Lagrange)

Dados n+1 puntos (x_i,y_i) con x_i distintos, el polinomio de grado <= n que los interpola es

P(x) = sum_(i=0..n) y_i prod_(j != i) (x-x_j)/(x_i-x_j).

En puntos 0,1,...,n, usar prefix/suffix para O(n) por evaluación. Newton: usar diferencias divididas, útil si se agregan puntos.

7.1 Series de potencias

- Geométrica: sum_(n >= 0) x^n = 1/(1-x). Derivada: $sum_(n >= 1) n x^n = x/(1-x)^2$.
- Formal: definir F(x) = sum a_n x^n; suma y producto por convolución ordinaria.
- [x^n]F(x) = F^((n))(0)/n! = (1/(2pii)) oint F(z)/z^(n+1) dz (Cauchy).

7.2 Teorema de Abel

Si sum a_n converge con suma S, entonces $lim_(x -> 1^-) sum a_n x^n = S$. (En sentido inverso, requiere condiciones de Tauber.) **Fórmula de sumación de Abel:**

sum_(k=1..n) a_k b_k = A_n b_n + sum_(k=1..n-1) A_k(b_k - b_(k+1)), A_k = sum_(i=1..k) a_i.

Análoga a integración por partes, útil para convergencia condicional.

7.3 Convoluciones

- Ordinaria: (f*g)(n) = sum_(i=0..n) f(i)g(n-i); OGF multiplica.
- Dirichlet: (f*g)(n) = sum_(d | n) f(d)g(n/d); multiplicativa si f,g lo son.
- Möbius: g = f * 1 <=> f = g * mu.
- gcd-convolución: (f *_gcd g)(n) = sum_(gcd(i,j)=n) f(i)g(j); calcular con zeta de múltiplos F(n) = sum_(n | d) f(d) y luego Möbius de múltiplos.
- lcm-convolución: análogo con zeta de divisores F(n) = sum_(d | n) f(d).

7.4 Newton generalizado y EGF

(1+x)^alpha = sum_(n >= 0) C(alpha,n) x^n. Para secuencias, la función generatriz exponencial (EGF) sum a_n x^n/n! convierte convolución binomial en producto; útil para Stirling, Bell, particiones etiquetadas.

7.5 Sumas hipergeométricas

Un término t_k es hipergeométrico si t_(k+1)/t_k es función racional de k (ej. C(n,k)).

- Pascal: C(n,k) = C(n-1, k-1) + C(n-1, k).
- Gosper: encuentra z_k tal que t_k = z_(k+1)-z_k, sumando sum_(k=a..b) t_k = z_(b+1)-z_a. Si no existe, la suma no es hipergeométrica.
- Zeilberger (creativa telescópica): para sum_k F(n,k) hipergeométrica en n, encuentra recurrencia lineal en n. Base de hyper en Mathematica.
- Chu-Vandermonde: sum_k C(r,k) C(s, n-k) = C(r+s, n). Variante: sum_k C(r,k) C(s, n+k) = C(r+s, r-n).
- Suma de Gauss: sum_k C(a,k) C(b,k) = C(a+b, a).

7.6 Algoritmos sobre series

- FFT/NTT para producto de polinomios en O(n log n).
- Potencias de series (Brent-Kung): F(x)^k mod x^N en O(N log N) con NTT + log/exp formales.
- Exp/log/raíz formal: derivar, integrar, resolver por Newton.
- Inversión de series: recursión de Newton, O(N log N).

8.1 Tipos y definición

Dada una sucesión (a_n)_(n >= 0), se define:

- OGF (ordinaria): A(x) = sum_(n >= 0) a_n x^n.
- EGF (exponencial): A^(x) = sum_(n >= 0) a_n x^n / n!.
- Dirichlet: D_A(s) = sum_(n >= 1) a_n / n^s (útil para convolución de Dirichlet).
- Ordinaria truncada: A_N(x) = sum_(n=0..N) a_n x^n (lo que manipulamos mód x^(N+1)).

Relación OGF/EGF: la OGF de a_n/n! es la EGF de a_n; la EGF de a_n * n! es la OGF de a_n. En CP, si hay etiquetas (permutaciones, particiones de conjuntos, árboles etiquetados) se usa EGF; si no, OGF.

8.2 Operaciones básicas

Con A(x) = sum a_n x^n, B(x) = sum b_n x^n:

- Suma: A+B <-> a_n+b_n.
- Producto (convolución): $(A * B)(x) <-> c_n = sum_(i=0..n) a_i b_(n-i)$.
- Derivada: A'(x) = sum_(n >= 1) n a_n x^(n-1) <-> n a_n.
- Integral: int_0^x A(t)dt = sum_(n >= 0) (a_n/(n+1)) x^(n+1).
- Desplazamiento: x^k A(x) <-> a_(n-k) (con a_(<0)=0).
- Escalamiento: A(cx) <-> c^n a_n.
- Producto por $x/(1-x)$: A(x)/(1-x) <-> sum_(i=0..n) a_i (prefijos).
- Producto por $1/(1-x^k)$: inserta bloques de tamaño k.
- Producto por $(1-x)$: A(x)(1-x) <-> a_n - a_(n-1) (diferencias).

8.3 Extracción de coeficientes

- [x^n]A(x) = A^((n))(0)/n! (Taylor).
- Cauchy (contorno): [x^n]A(x) = (1/(2pii)) oint_gamma A(z)/z^(n+1) dz.
- Extracción racional: si A(x) = P(x)/Q(x) con deg P < deg Q, descomponer en fracciones parciales P(x)/Q(x) = sum_j sum_(r=1..m_j) alpha_(j,r) / (1-lambda_j x)^r, y usar [x^n](1-lambda x)^(-r) = C(n+r-1, r-1) lambda^n. Esto resuelve cualquier recurrencia lineal homogénea con coeficientes constantes.

8.4 Series notables (tabla de referencia)

- 1/(1-x) = sum x^n
- 1/(1-x)^k = sum C(n+k-1, k-1) x^n
- (1+x)^alpha = sum C(alpha,n) x^n
- e^x = sum x^n/n!, ln(1+x) = sum_(n >= 1) ((-1)^(n-1)/n) x^n
- sin x = sum (-1)^n x^(2n+1)/(2n+1)!, cos x = sum (-1)^n x^(2n)/(2n)!
- x/(e^x-1) = sum B_n x^n/n! (Bernoulli)
- prod_(k >= 1) 1/(1-x^k) = sum p(n)x^n (particiones)
- prod_(k >= 1)(1+x^k) = sum q(n)x^n (particiones en partes distintas)
- 1/(1-x-x^2) = sum F_(n+1)x^n (Fibonacci)
- (1-x-sqrt(1-2x-3x^2))/(2x^2) = sum M_n x^n (Motzkin)
- C(x) = (1-sqrt(1-4x))/(2x) = sum C_n x^n (Catalan)

8.5 Funciones generatrices clásicas derivadas

- Catalan: C(x) = 1+xC(x)^2 => C(x) = (1-sqrt(1-4x))/(2x).
- Fibonacci: F(x) = x/(1-x-x^2). Binet: F_n = (phi^n-psi^n)/sqrt5.
- Bell (EGF): sum B_n x^n/n! = exp(e^x-1).
- Stirling 2a (EGF en n, OGF en k): $sum_(n >= k) S(n,k) x^n/n! = (e^x-1)^k / k!$.
- Stirling 1a sin signo (EGF): $sum_(n >= k) c(n,k) x^n/n! = (-ln(1-x))^k / k!$.
- Derangements (EGF): sum D_n x^n/n! = e^(-x)/(1-x).
- Eulerianos: sum_(n >= 0) A(n,k) x^n/n! = x^(k+1)/(1-x)^(k+1) (como serie en x).
- Particiones: P(x) = prod_(k >= 1) 1/(1-x^k). Pentagonal: $prod(1-x^k) = sum_j (-1)^j x^(j(3j-1)/2)$.

8.6 Composición, inversa y Lagrange

Si A(x) = sum a_n x^n con a_0=0, a_1 != 0, existe la **inversa composicional** A^(<-1>)(x) tal que A(A^(<-1>)(x)) = x.

Inversión de Lagrange. Si A(x) = x phi(A(x)) con phi(0) != 0, entonces [x^n]A(x) = (1/n)[t^(n-1)]phi(t)^n.
Equivalentemente, si w = x phi(w):
[x^n]f(w(x)) = (1/n)[t^(n-1)]f'(t)phi(t)^n.

Uso típico: contar árboles (Cayley: n^(n-2)), obtener coeficientes de la inversa de una serie dada, resolver ecuaciones funcionales A = x e^A (árboles etiquetados) o A = x(1+A)^k (Fuss-Catalan).

8.7 Composición y potencias de series (algoritmia)

Trabajando mód x^N:

- Inversa multiplicativa 1/A (con a_0 != 0): recursión de Newton $B_(2k) = B_k(2-AB_k), O(N log N)$.
- Logaritmo formal: log A = int A'/A.
- Exponencial formal: exp A por Newton: B_(2k) = B_k(1-log B_k + A).
- Raíz k-ésima: A^(1/k) = exp((1/k) log A).
- Potencia general: A^alpha = exp(alpha log A) (requiere a_0=1).
- Composición A(B(x)): O(N^(1.5)) con bloque de Brent-Kung; rara en CP, pero existe.

8.7 Composición y potencias de series (algoritmia) (cont.)

Todas usan NTT/FFT para O(N log N) por operación.

8.8 Ecuaciones funcionales y coeficientes

Método general: si A(x) = F(x, A(x)) con F analítica:

1. Escribir A = a_0 + a_1 x + ... e igualar coeficientes de x^n.
2. Si A = xphi(A): aplicar Lagrange.
3. Si A = F(A/x): sustituir B = A/x.
4. Si A = 1+xA^k: Fuss-Catalan, a_n = C(kn, n) / ((k-1)n+1).

8.9 Bivariadas y extracción diagonal

Para sucesiones a_(n,k), usar A(x,y) = sum a_(n,k)x^n y^k.

- Diagonal: sum a_(n,n)x^n = [y^0]A(x/y,y). Extraíble con residuos.
- Coeficiente binomial: sum_k C(n,k)x^k y^(n-k) = (x+y)^n.
- Delannoy: D(x,y) = 1/(1-x-y-xy).

8.10 Convoluciones como GFs

- Ordinaria: producto A(x)B(x).
- Binomial: sum_k C(n,k)a_k b_(n-k) <-> producto de EGF.
- Dirichlet: producto de series de Dirichlet, $(f*g)(n) = sum_(d | n) f(d)g(n/d)$.
- Möbius: g = f * 1 <=> f = g * mu; en GFs de Dirichlet: G = zeta * F, F = G/zeta = G * (1/zeta), con 1/zeta(s) = sum mu(n)n^(-s).
- Euler transform: prod_(k >= 1)(1-x^k)^(-b_k) = sum a_n x^n, con a_n = (1/n) sum_(k=1..n)(sum_(d | k) d b_d)a_(n-k). Inversa: $log A = sum_k b_k sum_(j >= 1) x^(kj)/j$.

8.11 Series de Fourier y extracción (puente)

La EGF/OGF a veces se extrae vía transformada de Fourier discreta (raíces n-ésimas de la unidad):

a_n = (1/N) sum_(j=0..N-1) omega^(-nj) A(omega^j), omega = e^(2pii/N), N>n.

Base de la multiplicación con NTT y del multipoint evaluation. **Fourier compleja:** f(x) = sum_(n in Z) c_n e^(inx), $c_n = (1/(2pi)) int_(-pi)^pi f(x)e^(-inx)dx$.

9.1 Definiciones básicas

- Grupo (G,*): asociativa, neutro e, inversos. |G|: cardinalidad del grupo.
- Orden de un elemento: ord(g) = min{n>0 : g^n = e}; siempre $ord(g) | |G|$.
- Subgrupo H <= G: cerrado bajo * e inversos.
- Cíclico: G = <g>. Z_n es el modelo.
- Generadores: g genera G <=> ord(g) = |G|. #generadores de $Z_n = phi(n)$.

9.2 Teoremas

Lagrange. H <= G finito => |H| | |G|. En particular ord(g) | |G|.

Orden primo. |G| = p primo => G ~= Z_p.

Unidades. (Z/nZ)^* cíclico <=> n in {1,2,4,p^k,2p^k}. Si sí, tiene phi(phi(n)) generadores.

9.3 Acción de grupo, órbita-estabilizador

G finito actúa sobre X. Orb(x) = {g * x : g in G}, $Stab(x) = {g : g * x = x}$.

|Orb(x)| * |Stab(x)| = |G|.

9.4 Lema de Burnside

Burnside. Sea G finito actuando sobre X. El número de órbitas es |X/G| = (1/|G|) sum_(g in G) |X^g|.
donde |G| es la cardinalidad del grupo (número de simetrías consideradas). X^g = {x in X : g * x = x} el conjunto de puntos fijos de g, y X/G el conjunto de configuraciones distintas salvo simetría.

Ejemplo (Z_n rotaciones): collares de n cuentas con k colores,

= (1/n) sum_(d | n) phi(d) k^(n/d).

9.5 Pólya (Burnside con pesos)

Sea j_i(g) = #ciclos de longitud i de la permutación de X inducida por g. Índice de ciclos:

Z(G; x_1,...,x_n) = (1/|G|) sum_(g in G) prod_(i=1..n) x_i^(j_i(g)).

Coloraciones con k colores: Z(G; k,...,k).

- Z_n: Z = (1/n) sum_(d | n) phi(d) x_d^(n/d).
- D_n (2n elementos):
  Z(D_n) = (1/2)Z(Z_n) + { (1/2)x_1 x_2^((n-1)/2) si n impar;
  (1/4)(x_1^2 x_2^((n-2)/2) + x_2^(n/2)) si n par }.

10.1 Vectores

u = (u_x, u_y); |u| = sqrt(u_x^2+u_y^2). Punto: $u * v = u_x v_x + u_y v_y = |u||v|cos theta. Cruz 2D: u x v = u_x v_y - u_y v_x = |u||v|sin theta. Orientación de A,B,C: sgn((B-A)x(C-A))$.

10.2 Áreas

- Shoelace: A = (1/2)|sum_i (x_i y_(i+1) - x_(i+1) y_i)|.
- Pick: A = I + B/2 - 1 (vértices en Z^2, polígono simple).
- Triángulo: A = (1/2)|(B-A)x(C-A)|.

10.3 Intersecciones y distancias

- Segmentos: AB cap CD != empty si $sgn(AB x AC) * sgn(AB x AD) <= 0 y análogo con CD$ (manejar colinealidad aparte).
- Punto-recta (A,B): d = |(B-A)x(P-A)| / |B-A|.
- Punto-segmento: proyectar t = ((P-A)*(B-A)) / |B-A|^2, clamp $t in [0,1]$.

10.4 Círculos

(x-c_x)^2 + (y-c_y)^2 = r^2. Intersección de dos círculos si $|r_1-r_2| <= d <= r_1+r_2 (d$ distancia entre centros); área vía dos segmentos circulares (fórmula de la *lens*).

10.5 Convex hull

- Monotone chain (Andrew): ordenar por (x,y), construir cadena inferior y superior; O(n log n).
- Graham scan: ordenar por ángulo, pila; O(n log n).

11.1 Esperanza, varianza y desigualdades

- E[X] = sum_x x Pr(X=x).
- Linealidad: E[X+Y] = E[X]+E[Y] siempre.
- Independencia: E[XY] = E[X]E[Y].
- Varianza: Var(X) = E[X^2]-E[X]^2; Var(aX+b) = a^2 Var(X); Var(X+Y) = Var X + Var Y si indep.
- Desigualdades: Markov Pr(X >= a) <= E[X]/a; Chebyshev $Pr(|X-mu| >= ksigma) <= 1/k^2$.

12.1 Eliminación gaussiana

Resolver Ax=b en O(n^3) con pivoteo parcial. det A = producto de pivotes (signo según permutaciones de filas). A^(-1) por Gauss-Jordan.

12.2 Cayley y Matrix-Tree

Cayley. #árboles etiquetados sobre n vértices: n^(n-2).

Matrix-Tree. Para grafo no dirigido con laplaciano L = D-A. #árboles generadores = det de cualquier menor principal (n-1)x(n-1) de L.

12.3 Regla de Simpson

int_a^b f(x)dx ~= ((b-a)/6)[f(a) + 4f((a+b)/2) + f(b)].

Compuesta con n par, h = (b-a)/n:

int_a^b f ~= (h/3)[f_0 + 4f_1 + 2f_2 + 4f_3 + ... + 4f_(n-1) + f_n].

12.4 Números armónicos

H_n = sum_(k=1..n) 1/k; H_n = ln n + gamma + O(1/n), $gamma ~= 0.5772. Usos: sum_(k=1..n) C(n,k)/k tiene forma cerrada con H_n$; análisis de algoritmos (quicksort: 2H_n-2). Identidad: $sum_(k=1..n) 1/(k(k+1)) = 1 - 1/(n+1)$.

12.5 Series de Fourier (para coeficientes)

- Trigonométrica: $f(x) = a_0/2 + sum_(n >= 1)(a_n cos nx + b_n sin nx), a_n = (1/pi) int_(-pi)^pi f(x)cos(nx)dx, análogo b_n$.
- Compleja: f(x) = sum_(n in Z) c_n e^(inx), $c_n = (1/(2pi)) int_(-pi)^pi f(x)e^(-inx)dx$.
- Ortogonalidad: (1/(2pi)) int_(-pi)^pi e^(i(n-m)x)dx = delta_(nm).